Subtyping Algebraic Protocols

ANONYMOUS AUTHOR(S)

ABSTRACT: subtyping AND interoperability

ACM Reference Format:
Anonymous Author(s). 2026. Subtyping Algebraic Protocols. 1, 1 (July 2026), 49 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Session types [28, 29, 51] have become a quasi-standard for defining stateful communication protocols and reasoning about them. A (binary) session type specifies a protocol as a set of sequences of typed messages going forth and back between two participants. Many interesting protocols are recursive, which requires nontrivial reasoning about equivalence and subtyping of session types. For context-free session types [52], equivalence is expensive to decide and subtyping is undecidable. This paper studies algebraic representations of session types that admit linear time equivalence and subtyping while retaining most of the expressivity of context-free session types.

Traditionally, protocols are modeled using equirecursive session types like `IntListS` (session type for sending finitely many integers and then close) and `IntStreamS` (send infinitely many integers) that are assigned to one end of a communication channel:

```
type IntListS   = μα. ⊕{Cons: !Int.α, Nil: End} -- finite sequences
type IntStreamS = μα. !Int.α                    -- infinite sequence
```

In session types, the $\oplus\{l_i : S_i\}$ constructor stands for selection: send a label l_i, like `Cons` or `Nil`, and continue according to the chosen branch S_i; the $!T.S$ constructor stands for sending a value of type T and continuing the protocol according to S. The other end of the channel has the *dual type* where sending and receiving ($?T.S$) is flipped, as well as selection and branching indicated by $\&\{l_i : S_i\}$: if we receive a label l_i, then continue with S_i.

Using recursive types directly is somewhat tedious as equivalence of equirecursive types[1] amounts to comparing infinite regular trees or reasoning up to unfolding of the recursive types. While equivalence is decidable in $O(n^2)$ for traditional, tail-recursive session types (where n corresponds to the size of the type), the difficulty increases when considering context-free session types [4, 17, 41, 42, 52] where the first equivalence test was doubly exponential[3], though recent work indicates it might be polynomially decidable in $O(n^{12})$ [47].

These results expose a tension between expressivity and tractability: richer session type languages lead to expensive equivalence and undecidable subtyping. Our goal is to identify a representation that retains expressivity close to context-free session types while admitting efficient equivalence and subtyping.

Prior work proposed parameterized algebraic protocols [38], where recursive protocols are defined analogously to algebraic datatypes (ADTs). A protocol is defined by a nominal type whose

[1] An equirecursive type is considered *equal* to its unfoldings.

ACM XXXX-XXXX/2026/7-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

constructors describe the possible next actions. In this framework, we can define `IntListP`, the algebraic protocol analogous to `IntListS`, and `IntStreamP`, analogous to `IntStreamS`, as follows:

```
protocol IntListP   = Nil | Cons Int IntListP
protocol IntStreamP = Next Int IntStreamP
```

As with ADTs, the combination of recursive types with sum-of-product types is "shrink wrapped" into a nominal type. In consequence, deciding equivalence of protocol types takes time linear in the size of the type, regardless of whether the underlying protocol is tail-recursive or context free.

The prior work gave an informal outline of an embedding of algebraic protocols into context-free session types, but it did not offer a comparison with traditional, tail-recursive session types. This paper fills this gap.

1.1 Traditional Session Types to Algebraic Protocols

We now give an outline of a translation from traditional session types (TST) to algebraic session types (AST). The translation replaces equirecursive types by explicit systems of recursive equations. The main difficulty in translating a program with TST to AST lies in managing the unfolding of equirecursive types. That is, at every point in a TST typing derivation we may silently apply a type conversion that unrolls a recursive session type as in $\mu\alpha.S \approx S[\mu\alpha.S/\alpha]$. Our translation to AST proceeds as follows. First, we translate *all* types in a TST program into a system of recursive type equations where all left hand sides are type variables and equations are flat, i.e., they consist of a type constructor applied to type variables as in $\alpha \rightarrow \beta$ or $?\alpha.\beta$. All type variables occurring on right hand sides must be defined by a single equation. Defined type variables on the left hand sides may be used zero or more times on the right hand sides. Here are two equation systems (one line each) corresponding to our running examples.

```
IntStreamS   α = !β.α                    β = Int
IntListS     α = (+){Cons: β, Nil: δ}    β = !γ.α     γ = Int    δ = End
```

Such an equation system can be considered as a labeled transition system (LTS). Each type variable corresponds to a state and each equation induces outgoing labeled transitions. For example, in the LTS corresponding to `IntListS`, there are labeled transitions $\alpha \xrightarrow{+Cons} \beta$ and $\alpha \xrightarrow{+Nil} \delta$ and further transitions for β, γ, and δ. Generally, the labeled transitions indicate the top-level type constructor and the particular step taken. In a second step, we minimize this LTS. Minimization corresponds to equating bisimilar type variables, where bisimilarity corresponds to type equivalence of equirecursive types. Thus, after minimization two types are bisimilar if and only if they are represented by the *same* type variable. Consequently, we can eliminate the conversion rule from typing as two types are now equivalent iff they are represented by the same type variable.

The final step of the translation reads algebraic protocols from the equations. To this end, we generate a decision protocol for each type variable with a select type or branch type (like α for `IntListS`) and use predefined protocols for sequence, empty protocol, and repetition to translate sending or receiving session types. For `IntListS` and `IntStreamS`, our translation yields the algebraic protocols `IntListSP` and `IntStreamSP`, shown after the definitions of the auxiliary protocols.

```
protocol Seq a b = Seq a b                -- first a then b
protocol Eps = Eps                        -- empty protocol
protocol Repeat a = Repeat a (Repeat a) -- infinitely often a
protocol List a = Cons a (List a) | Nil -- select/branch: Cons or Nil

IntListSP   = List (Seq Int Eps)
IntStreamSP = Repeat (Seq Int Eps)
```

Here the `List` protocol is the only custom protocol generated by the translation. The other protocols `Seq`, `Eps`, and `Repeat` are predefined.

The resulting algebraic protocols are bisimilar to the original session types when both are interpreted as labeled transition systems. We consider this outcome as a statement of *interoperability* between the original TST program and its AST translation. Interoperability results from an innocuous change that was hinted at, but not executed in the prior work [38]: single constructor protocols do not send or receive their constructor. That is, the protocols `Seq`, `Eps`, and `Repeat` are tag-free as they do not transmit a tag. In our example, it even holds that all three representations of the types are bisimilar.

```
IntListS   ~ IntListSP   ~ IntListP
IntStreamS ~ IntStreamSP ~ IntStreamP
```

1.2 Subtyping for Algebraic Protocols

We define a subtyping relation on parameterized algebraic protocols. This subtyping relation is inspired by prior work on constructor subtyping [8], where a subtype of an algebraic data type is defined by a non-empty subset of its constructors. In our framework, the protocol `IntListP` comes in three variants: `IntListP[Nil]`, `IntListP[Cons]`, and `IntListP[Nil,Cons]` (= `IntListP`). They roughly correspond to the session types $\oplus${`Nil: End`}, $\mu\alpha.\oplus${`Cons: !Int.`α}, and `IntListS`. To define the subtyping relation on these variants, we have to give the protocol a *direction*: the type `!IntListP` interprets the protocol in the forward direction (corresponding exactly to `IntListS`), whereas `?IntListP` interprets the protocol in the reverse direction (corresponding exactly to the dual of `IntListS`). Taking directionality into account, we obtain the following subtyping judgments:

```
!IntListP <: !IntListP[Nil]          ?IntListP[Nil]  <: ?IntListP
!IntListP <: !IntListP[Cons]         ?IntListP[Cons] <: ?IntListP
```

The full subtyping relation corresponds to the reflexive transitive closure of the relation defined by these judgments.

The subtyping relation resulting from constructor subtyping on algebraic protocols is still decidable in linear time in the size of the type if comparing two sets of constructors takes constant time[2]. In comparison, subtyping for tail-recursive session types is decidable in $O(n^2)$ and subtyping for context-free session types is undecidable [42]. That is, algebraic protocols with constructor subtyping carve out a subset of context-free session types with an efficiently decidable subtyping relation.

1.3 Traditional Session Types with Subtyping to Algebraic Protocols

The constructor subtyping relation is key in extending the mapping from traditional, tail-recursive session types to algebraic protocols to a setting with subtyping. The first step in constructing this mapping is just like in Section 1.1: we represent all types in a program by a system of type equations. The second step is also the same: we minimize the equations so that each type variable represents an equivalence class of bisimilar types.

In our model, a type is a pair $\langle \alpha, \mathcal{E} \rangle$ where $\mathcal{E}$ is a system of type equations and α is the left hand side of an equation in $\mathcal{E}$. To stay in this framework, we define subsumption on type equations: Subsumption keeps the variable α; it only adapts $\mathcal{E} <: \mathcal{E}'$ where the standard subsumption may be applied to any suitable equation. For instance, here is the subsumption from `IntListS` to `IntListS[Nil]` which eliminates the constructor `Cons`:

[2]This assumption is reasonable because the number of constructors is usually bounded by a small constant and a set of constructors can be represented by a bit set.

```
IntListS          α = (+){Cons: β, Nil: δ}    β = !γ.α     γ = Int    δ = End
IntListS[Nil]     α = (+){Nil: δ}             β = !γ.α     γ = Int    δ = End
```

The domains of $\mathcal{E}$ and $\mathcal{E}'$ coincide, i.e., all left hand sides stay the same and we do not drop unreachable type variables like β and γ. Getting from the right hand side of α in the first line to the second line consists in applying the standard subsumption rule for selection in functional session types [24]. In the general case, we may apply subsumption to each equation individually, taking care whether the left hand side variable occurs in covariant (like in our example) or in contravariant position.

Contributions

This paper develops algebraic protocols with constructor subtyping and shows that they provide an efficient and expressive framework for typing session-based programs. Algebraic protocols identify a tractable fragment of context-free session types with linear time algorithms for equivalence and subtyping.

(1) We define a subtyping relation for algebraic protocol types based on constructor subtyping. Subtyping is obtained by selecting non-empty subsets of protocol constructors. The symmetric kernel of this relation coincides with the equivalence relation defined in prior work [38]. Subtyping and type equivalence are decidable in linear time.
(2) We define the calculus AlgST$^{<:}$, a core language based on linear System F with non-linear recursion and session types expressed as algebraic protocols. It extends AlgST from prior work with constructor subtyping. Typing rules are presented in bidirectional form and admit a direct algorithmic interpretation.
(3) We establish type soundness for AlgST$^{<:}$ via preservation and progress.
(4) We define a translation from traditional tail-recursive session types to AlgST. The translation preserves typing and establishes a reduction correspondence.
(5) We extend this translation to session types with subtyping under a restriction on the representation of session types as recursive equation systems.
(6) We provide an implementation of the system as a publicly available artifact, including practical extensions described in Section 6.

Overview

TODO

2 Algebraic Protocols, Session Types, and Subtyping

This section starts with a brief introduction to algebraic protocols driven by examples. Next, it motivates constructor subtyping via compatibility for evolving interfaces. The remaining sections discuss the translation from TST (traditional, tail-recursive session types) to algebraic session types with examples, first without subtyping and finally with subtyping. We use a syntax inspired by Haskell, as implemented in our artifact. Examples make liberal use of practical extensions of the formal system (cf. Section 6).

To avoid confusion, we tag each traditional session type name with a trailing `S` and each protocol type name with a trailing `P`.

NOTE: we are going to switch to a structural notation for constructor subtyping. That is, types can have arbitrary names, but the subtyping relation is determined by their constructor signatures.

2.1 Simple Algebraic Protocols

We start with a brief recap of algebraic protocols as presented by Mordido et al. [38]. As a running example, we choose the simple arithmetic server, which is a standard example from the session type literature (cf. [22]). The arithmetic server accepts two commands, `Neg` and `Quit`. When it receives the `Neg` command, it waits for an integer argument, responds with an integer (the result) and terminates. When it receives `Quit`, it terminates the connection directly.

Here is the type of this server adapted from the literature.

```
Arith0S = &{Neg:?Int.!Int.end, Quit:end}
```

The corresponding algebraic protocol `Arith0P` looks as follows.

```
protocol Arith0P = Quit | Neg Int -Int
```

Unlike the session type `Arith0S`, `Arith0P` is just a *protocol*. Here is the crucial difference: The session type `Arith0S` already fixes the direction of communication, as the operators `&` and `?` indicate receiving. The protocol `Arith0P` only fixes the structure of the communication, i.e., `Neg` followed by an `Int` transmission in the same direction while the `-Int` indicates a transmission in the opposite direction. We materialize the direction of a protocol by using it in a session type as in `?Arith0P.End`. This algebraic session type corresponds exactly to `Arith0S`.

The server implementation would match on the command as usual, but with the following types dictated by the theory of algebraic protocols.

```
c : ?Arith0P.s ⊢ match c with { Quit c → ...   -- c : s
                                Neg c → ... } -- c : ?Int.!Int.s
```

The actual implementation of the server is straightforward.

```
arith0Server : ?Arith0P.end → unit
arith0Server c = match c with {
    Quit c → close c,
    Neg c  → let (x, c) = receiveInt [!Int.end] c in
             let c = sendInt [end] (0-x) c in
             close c }
```

The type arguments of `receiveInt` and `sendInt` (in square brackets) are the types of the respective continuation channels. They specify the type of the variable `c` that is bound in the respective let.

A client implementation would use the select operator as usual, but with the following types prescribed by algebraic protocols. This behavior is dual to the one for matching.

```
select Quit [s] : !Arith0P.s → s
select Neg [s]  : !Arith0P.s → !Int.?Int.s
```

A potential client for the protocol would look like this.

```
arith0Client : !Arith0P.end → Int → Int
arith0Client c x =
  let c = select Neg [end] c in
  let c = sendInt [?Int.end] x c in
  let (r, c) = receiveInt [end] c in
  let _ = close c in
  r
```

2.2 Simple Constructor Subtyping

Consider an extension of the arithmetic server with an additional operation indicated by the command `Add`. The traditional session type changes to include the new alternative.

```
Arith0S' = &{Add:?Int.?Int.!Int.end, Neg:?Int.!Int.end, Quit:end}
```

The standard subtyping for session types yields `Arith0S <: Arith0S'` because a channel of type `Arith0S` can be processed by code that expects the supertype `Arith0S'`: The code has an additional branch for `Add` that is never exercised.

The corresponding algebraic protocols are

```
protocol Arith0P  = Quit | Neg Int -Int
protocol Arith0P' = Quit | Neg Int -Int | Add Int Int -Int
```

With constructor subtyping for algebraic protocols, we can write `Arith0P'[Neg,Quit]` to select the constructors `Neg` and `Quit` in `Arith0P'`. This way, `Arith0P'` is a supertype of `Arith0P'[Neg,Quit]`, which corresponds exactly to the previous protocol `Arith0P` from Section 2.1. On the algebraic session type level, we obtain `?Arith0P'[Neg,Quit].end<:?Arith0P'.end` and, at the client end, `!Arith0P'.end<:!Arith0P'[Neg,Quit]<:!Arith0P'[Neg]`. The last subtyping judgment shows that we can reuse our client implementation with an extended server as `arith0Client` is already typable with `!Arith0P[Neg].end`.

On the expression level, we extend the match expression in the server by the case for the `Add` command, just like we would for `Arith0S'`.

2.3 Recursive Algebraic Protocols and Subtyping

Next, we move to the recursive version of the arithmetic server. It repeatedly accepts commands before quitting as indicated by its traditional session type.

```
ArithS = μα.&{Neg:?Int.!Int.α, Quit:end}
```

The corresponding protocol becomes recursive, too.

```
protocol ArithP = Quit | Neg Int -Int ArithP
```

The implementations of client and server are omitted because they change insignificantly with respect to Section 2.1.

Instead, we focus on the extension of `ArithS` with a new recursive case for `Add` as follows.

```
ArithS' = μα.&{Add:?Int.?Int.!Int.α,Neg:?Int.!Int.α, Quit:end}
```

The standard subtyping for recursive session types yields `ArithS <: ArithS'` because a channel of type `ArithS` can be processed by code that expects the supertype `ArithS'`: The code has an additional branch for `Add` that is never exercised.

The corresponding algebraic protocols are

```
protocol ArithP  = Quit | Neg Int -Int ArithP
protocol ArithP' = Quit | Neg Int -Int ArithP' | Add Int Int -Int ArithP'
```

With constructor subtyping for algebraic protocols, we can write `ArithP'[Neg,Quit]` to select the constructors `Neg` and `Quit` in `ArithP'`. This way, `ArithP'` is a supertype of `ArithP'[Neg,Quit]`, which corresponds exactly to the previous protocol `ArithP` from Section 2.1.

On the expression level, we extend the match expression in the server by the case for the `Add` command, just like we would for `ArithS'`.

The subtyping `ArithS <: ArithS'` is decidable by an $O(n^2)$ algorithm, because the session types involved are traditional (i.e., tail-recursive). However, the following protocol for serializing abstract syntax trees is not tail-recursive:

```
protocol AstP = Const Int | Add AstP AstP | Mul AstP AstP
```

Nevertheless, constructor subtyping immediately yields subtypes `AstP[Const]`, `AstP[Const,Add]`, and `AstP[Const,Mul]` of `AstP`[3]. This is remarkable because the non-tail-recursive nature of the protocol makes it impossible to use the traditional algorithm to check subtyping of the corresponding (context-free) recursive types:

```
AstS1 = μα.&{Const: ?Int, Add: α;α, Mul: α;α}
AstS2 = μα.&{Const: ?Int, Add: α;α}
```

The current knowledge about subtyping for context-free session types is that the general problem is undecidable [42], but there exists a sound semi-decision procedure that seems to work satisfactorily in practice [48]. In our setting, subtyping is decidable in linear time.

OLD-OLD-OLD

The main feature of algebraic protocols that we did not touch, yet, is parametric protocols. That means that protocols can be parametric over other protocols. This feature enables the definition of protocols like `Seq` from the introduction. We introduce it in XXX along with the discussion of the translation.

2.4 From Session Types to Algebraic Protocols

Starting from this section we discuss the translation from traditional, tail-recursive session types to algebraic protocols using the implementation of `arithServer` in Section 2.1 as a running example. This code does not use subtyping.

To express all types occurring in `arithServer`, we need the following system of type equations.

$$
\begin{aligned}
\alpha_1 &= \&\{Neg : \alpha_2, Quit : \alpha_3\} && \text{c}\\
\alpha_2 &= ?\alpha_4.\alpha_5\\
\alpha_3 &= \texttt{End}\\
\alpha_4 &= \texttt{Int}\\
\alpha_5 &= !\alpha_6.\alpha_1\\
\alpha_6 &= \texttt{Int}\\
\alpha_7 &= \alpha_1 \to \alpha_8 && \text{arithServer}\\
\alpha_8 &= \texttt{Unit}\\
\alpha_9 &= \alpha_3 \to \alpha_8 && \text{close}\\
\alpha_{10} &= \alpha_2 \to \alpha_{11} && \text{receiveInt}\\
\alpha_{11} &= \alpha_4 \times \alpha_5\\
\alpha_{12} &= \alpha_5 \to \alpha_1\\
\alpha_{13} &= \alpha_6 \to \alpha_{12} && \text{sendInt}
\end{aligned}
$$

For this system, typing rules have the form $\mathcal{E} \mid \Gamma \vdash e : \alpha$ where $\mathcal{E}$ is a system of flat type equations, Γ is a typing context mapping variables to type variables defined in $\mathcal{E}$, e is an expression, and α is a type variable. Here are some exemplary typing rules for variables, function application,

[3]Subtypes without the `Const` case can also be formed, but they are not very interesting.

and abstraction:

$$\mathcal{E} \mid \Gamma, x : \alpha \vdash x : \alpha \qquad \frac{\mathcal{E} \mid \Gamma \vdash e_1 : \alpha_1 \qquad \mathcal{E} \mid \Gamma \vdash e_2 : \alpha_2 \qquad \alpha_1 = \alpha_2 \to \alpha_3 \in \mathcal{E}}{\mathcal{E} \mid \Gamma \vdash e_1\, e_2 : \alpha_3}$$

$$\frac{\mathcal{E} \mid \Gamma, x : \alpha_2 \vdash e : \alpha_3 \qquad \alpha_1 = \alpha_2 \to \alpha_3 \in \mathcal{E} \qquad T \equiv \mathit{extract}(\mathcal{E}, \alpha_2)}{\mathcal{E} \mid \Gamma \vdash \lambda x{:}\, T.e : \alpha_1}$$

The typing rule for typed lambda abstraction has to extract the type from the equation system by traversing it. For each session type variable α_i, extraction inserts a $\mu\alpha_i$ binder and refers back to α_i if it is seen again on the traversal.

Any recursively typed term can be type checked using a system of recursive type equations, so nothing is lost in this step.

In our running example, minimization equates variables α_4 and α_6, both of which stand for the type `Int`. No further simplification is possible.

Next, we focus on the session type variables α_1, α_2, α_3, and α_5. We start with α_1 because it holds a branch type and invent a new protocol name, say, `NegQuit` with two constructors `Neg` and `Quit`. For the `Neg` constructor, we continue with $\alpha_2 = ?\alpha_4.\alpha_5$. As this session type has the same direction as branching, we extract its payload type α_4 (i.e., `Int`) and add it to the constructor with positive direction. Continuing with $\alpha_5 = !\alpha_6.\alpha 1$, we find a session type in the reverse direction, so we extract its payload from α_6 (i.e., `Int`) and add it to the constructor with negative direction. Continuing with α_1, we realize that we have seen this type variable before and conclude the constuctor with a positive `NegQuit` argument.

For the `Quit` constructor, we consider α_3 = `End`. However, we cannot add `End` as a protocol argument because it would mean that we wanted to transmit a *channel* of type *TEnd*, so we do not add it as an argument to the constructor, but rather finish the constructor without arguments.

The resulting protocol type is

```
protocol NegQuit = Neg Int -Int NegQuit | Quit
```

2.5 Discussion

Consider an extension of the arithmetic server with an additional operation indicated by the command `Add`. Here is an example that demonstrates the limitations of this approach to protocol subtyping.

```
protocol Msg = M1 | M2
protocol QR a b = Step a -b (QR a b) | Quit
```

The protocol `QR Msg Msg` indicates that client and server send `M1` and `M2` messages forth and back. If we evolve the message protocol with additional messages, then constructor subtyping will fail. Consider the extended message type

```
protocol Msg' = M1 | M2 | M3
```

Using `QR Msg' Msg'` means that the client can send an arbitrary message `M1`, `M2`, or `M3`, but it has to be prepared to handle the same messages as input. That is, even a client that promises to send only `M1` and `M2` has to be prepared to process a message `M3` in response. Hence, `?QR Msg'[M1,M2] Msg'` is a subtype of `?QR Msg' Msg'`, but `?QR Msg'[M1,M2] Msg'[M1,M2]` is not, because the second parameter of `QR` occurs under inversion (i.e., with revered direction). In short, we cannot obtain a type equivalent to `QR Msg Msg` as a subtype of `QR Msg' Msg'`.

```
protocol AB = A | B
protocol Msg'' a b = M1 -a | M2 -b
```

```
protocol Msg''' a b c = M1 -a | M2 -b | M3 -c

!Msg'' AB A <: !Msg'' AB AB
?Msg'' AB A <: ?Msg'' A A
```

3 Types

The type structure underlying AlgST comprises three parts: the kind structure, the type language and protocol definitions. The type language and protocol definitions are mutually recursive. To simplify the exposition, the formalized language is based on linear System F. While we formalize protocol definitions, we do not formalize the standard treatment of (linear) algebraic datatypes and pattern matching, although we use it in examples.

The type language encompasses types with different features (as illustrated in Section 2). For a correct type abstraction, we need to endow the system with a classification of types through kinds. AlgST distinguishes three kinds.

- **S** classifies session types, which in turn classify endpoints of communication channels.
- **T** classifies all types that classify run-time values.
- **P** classifies protocol types. The kind **P** subsumes any other kind by an implicit lifting that associates a type with a protocol suitable to transmit it. A protocol type per se classifies pure behavior, but no run-time values.

Kinds are linearly ordered in a subkinding relation, which is the reflexive transitive closure of the strict ordering $\mathbf{S} < \mathbf{T} < \mathbf{P}$. Type formation includes a standard kind subsumption rule.

Here is the grammar of types:

$$
\begin{array}{rcll}
S, T, U & ::= & \mathtt{Unit} \mid T \to U \mid T \otimes U \mid \forall \alpha : \kappa . T \mid \alpha \mid & \text{functional types} \\
& & \quad ?T.S \mid !T.S \mid \mathtt{End}_? \mid \mathtt{End}_! \mid \mathtt{Dual}\ S \mid & \text{session types} \\
& & \quad \rho\ \overline{T} \mid -T & \text{protocol types}
\end{array}
$$

We mostly use the metavariable S to range over session types (kind **S**). Metavariables T, U range over functional types (kind **T**) and protocol types (kind **P**). Functional types comprise the usual types of linear System F: unit, function, product, universal quantification, and type variables. The session type operators are receiving, sending, passive and active termination, and the dual operator. The dual operator swaps the direction of all communications in the spine of a session type. Thus, a session type is a sequence of communications either terminated by $\mathtt{End}_?$ or by $\mathtt{End}_!$, or extensible with a type variable α of kind **S**.

The protocol type $\rho\ \overline{T}$ refers to the protocol declared as ρ with parameters $\overline{T}$. The operator "$-$" reverses the direction of communication of its protocol parameter. While duality transforms session types outside-in, the reverse operator transforms protocols inside-out.

A kind context Δ maps type variables to kinds and protocol names to first-order arrow kinds that determine the number of parameters of the protocol. Formally, we should write $\varepsilon \to \mathbf{P}$ for the kind of a protocol constructor without parameters, but we often abbreviate it to **P**.

$$\Delta \ ::= \ \cdot \mid \Delta, \alpha : \kappa \mid \Delta, \rho : \overline{\mathbf{P}} \to \mathbf{P}$$

Figure 1 contains the type formation rules. We present them in algorithmic form from the beginning. The judgment $\Delta \vdash T \Rightarrow \kappa$ assigns to type T its minimal kind κ under kind context Δ. The corresponding checking judgment $\Delta \vdash T \Leftarrow \kappa$ takes inputs T and κ and checks if the synthesized kind is a subkind of the expected one.

The types for unit (T-Unit), functions (T-Arrow), pairs (T-Pair), and universal polymorphism (T-Poly) have kind **T** like their constituents. Variables have any kind that has been assigned to them

Type formation (synthesis, check-against) $\boxed{\Delta \vdash T \Rightarrow \kappa, \quad \Delta \vdash T \Leftarrow \kappa}$

$$\textsc{T-Unit}\ \frac{}{\Delta \vdash \mathtt{Unit} \Rightarrow \mathbf{T}} \qquad \textsc{T-Arrow}\ \frac{\Delta \vdash T \Leftarrow \mathbf{T} \qquad \Delta \vdash U \Leftarrow \mathbf{T}}{\Delta \vdash T \to U \Rightarrow \mathbf{T}} \qquad \textsc{T-Pair}\ \frac{\Delta \vdash T \Leftarrow \mathbf{T} \qquad \Delta \vdash U \Leftarrow \mathbf{T}}{\Delta \vdash T \otimes U \Rightarrow \mathbf{T}} \qquad \textsc{T-Var}\ \frac{\alpha : \kappa \in \Delta}{\Delta \vdash \alpha \Rightarrow \kappa}$$

$$\textsc{T-Poly}\ \frac{\Delta, \alpha : \kappa \vdash T \Leftarrow \mathbf{T}}{\Delta \vdash \forall \alpha : \kappa . T \Rightarrow \mathbf{T}} \qquad \textsc{T-In}\ \frac{\Delta \vdash T \Leftarrow \mathbf{P} \qquad \Delta \vdash S \Leftarrow \mathbf{S}}{\Delta \vdash ?T.S \Rightarrow \mathbf{S}} \qquad \textsc{T-Out}\ \frac{\Delta \vdash T \Leftarrow \mathbf{P} \qquad \Delta \vdash S \Leftarrow \mathbf{S}}{\Delta \vdash !T.S \Rightarrow \mathbf{S}}$$

$$\textsc{T-End?}\ \Delta \vdash \mathtt{End}_? \Rightarrow \mathbf{S} \qquad \textsc{T-End!}\ \Delta \vdash \mathtt{End}_! \Rightarrow \mathbf{S}$$

$$\textsc{T-Dual}\ \frac{\Delta \vdash S \Leftarrow \mathbf{S}}{\Delta \vdash \mathtt{Dual}\ S \Rightarrow \mathbf{S}} \qquad \textsc{T-Protocol}\ \frac{\rho : \overline{\mathbf{P}} \to \mathbf{P} \in \Delta \qquad \Delta \vdash \overline{T} \Leftarrow \overline{\mathbf{P}}}{\Delta \vdash \rho\ \overline{T} \Rightarrow \mathbf{P}} \qquad \textsc{T-MsgNeg}\ \frac{\Delta \vdash T \Leftarrow \mathbf{P}}{\Delta \vdash -T \Rightarrow \mathbf{P}} \qquad \textsc{T-Sub}\ \frac{\Delta \vdash T \Rightarrow \kappa \qquad \kappa \leq \kappa'}{\Delta \vdash T \Leftarrow \kappa'}$$

Fig. 1. Algorithmic type formation rules

(T-Var). AlgST's notion of session type distinguishes passive $\mathtt{End}_?$ and active $\mathtt{End}_!$ termination of a session (T-End?, T-End!), and the dual operator (T-Dual). Conventional session types require types for the values exchanged in messages; here we ask for protocols instead (T-In and T-Out). Rule T-Protocol builds a protocol type from a protocol name ρ. We assume that the kind context contains a binding for ρ that determines its arity. All parameters to a protocol must be protocols themselves. The latter constitutes no restriction because any type can be lifted to a protocol. A protocol type can change direction using the reverse operator (rule T-MsgNeg).

Algebraic protocol types $\rho\ \overline{\alpha}$ are defined analogously to algebraic datatypes.

$$\mathtt{protocol}\ \rho\ \overline{\alpha} = \{C_i\ \overline{T_i}\}_{i \in I}$$

This declaration defines the protocol type constructor ρ that takes as many parameters as indicated by $\overline{\alpha}$. The protocol has choices indexed by $i \in I$. Each choice is tagged by a *selector tag* C_i that guards a sequence of subprotocols $\overline{T_i}$, to be processed in that order. We assume that selector tags are globally unique. Program may use any full application $\rho\ \overline{U}$ as a protocol type. We check a single protocol declaration using the formation rules where Δ may refer to previously defined protocols:[4]

$$\frac{\mathtt{protocol}\ \rho\ \overline{\alpha} = \{C_i\ \overline{T_i}\}_{i \in I} \qquad \Delta, \rho : \overline{\mathbf{P}} \to \mathbf{P}, \overline{\alpha} : \overline{\mathbf{P}} \vdash \overline{T_i} \Leftarrow \mathbf{P} \qquad (\forall i \in I)}{\Delta \vdash \rho \Rightarrow \overline{\mathbf{P}} \to \mathbf{P}}$$

Types obey a kind-indexed conversion relation. Figure 2 presents the declarative rules. Conversion is reflexive, symmetric, and transitive (C-Refl, C-Sym, C-Trans). We define duality as a conversion on session types. The rules entail that duality changes the direction of transmissions from the outside (C-DualEnd?, C-DualEnd!, C-DualIn, C-DualOut) and that duality is involutory (C-DualInv). Kind subsumption is lifted to conversion (C-Sub). The last line governs the interaction between the reverse operator and the session type constructors: reverse flips the direction from the inside (C-NegIn, C-NegOut). Reverse is also involutory (C-NegInv). We omit the standard congruence rules.

[4]For mutually recursive protocols $\overline{\rho}$, we add the assumptions for all $\overline{\rho}$ when checking each individual ρ_i.

Type conversion $\boxed{\Delta \vdash T \equiv T : \kappa}$

$$\textsc{C-Refl}\ \frac{\Delta \vdash T \Leftarrow \kappa}{\Delta \vdash T \equiv T : \kappa} \qquad \textsc{C-Sym}\ \frac{\Delta \vdash T_1 \equiv T_2 : \kappa}{\Delta \vdash T_2 \equiv T_1 : \kappa} \qquad \textsc{C-Trans}\ \frac{\Delta \vdash T_1 \equiv T_2 : \kappa \qquad \Delta \vdash T_2 \equiv T_3 : \kappa}{\Delta \vdash T_1 \equiv T_3 : \kappa}$$

$$\textsc{C-DualEnd?}\ \Delta \vdash \mathtt{Dual}\ \mathtt{End}_? \equiv \mathtt{End}_! : \mathbf{S} \qquad \textsc{C-DualEnd!}\ \Delta \vdash \mathtt{Dual}\ \mathtt{End}_! \equiv \mathtt{End}_? : \mathbf{S}$$

$$\textsc{C-DualIn}\ \frac{\Delta \vdash T \Leftarrow \mathbf{P} \qquad \Delta \vdash S \Leftarrow \mathbf{S}}{\Delta \vdash \mathtt{Dual}\ (?T.S) \equiv\ !T.(\mathtt{Dual}\ S) : \mathbf{S}}$$

$$\textsc{C-DualOut}\ \frac{\Delta \vdash T \Leftarrow \mathbf{P} \qquad \Delta \vdash S \Leftarrow \mathbf{S}}{\Delta \vdash \mathtt{Dual}\ (!T.S) \equiv ?T.(\mathtt{Dual}\ S) : \mathbf{S}} \qquad \textsc{C-DualInv}\ \frac{\Delta \vdash S \Leftarrow \mathbf{S}}{\Delta \vdash \mathtt{Dual}\ (\mathtt{Dual}\ S) \equiv S : \mathbf{S}} \qquad \textsc{C-Sub}\ \frac{\Delta \vdash T_1 \equiv T_2 : \kappa \qquad \kappa \leq \kappa'}{\Delta \vdash T_1 \equiv T_2 : \kappa'}$$

$$\textsc{C-NegIn}\ \frac{\Delta \vdash T \Leftarrow \mathbf{P} \qquad \Delta \vdash S \Leftarrow \mathbf{S}}{\Delta \vdash ?(-T).S \equiv\ !T.S : \mathbf{S}} \qquad \textsc{C-NegOut}\ \frac{\Delta \vdash T \Leftarrow \mathbf{P} \qquad \Delta \vdash S \Leftarrow \mathbf{S}}{\Delta \vdash\ !(-T).S \equiv ?T.S : \mathbf{S}} \qquad \textsc{C-NegInv}\ \frac{\Delta \vdash T \Leftarrow \mathbf{P}}{\Delta \vdash -(-T) \equiv T : \mathbf{P}}$$

Fig. 2. Type conversion (congruence rules omitted)

While type formation is algorithmic, the declarative definition of type conversion is more perspicuous. For the upcoming algorithmic expression typing relation we define a normalization algorithm for types, which is sound and complete with respect to the declarative rules.

In a type in normal form, the reverse operator can only occur at most once at the top level (of a type of kind $\mathbf{P}$) and at the top level of the parameters of a protocol type. The `Dual` operator only appears on type variables, at the end of the spine of a session type. Thus an algorithm to compute normal forms must push all `Dual` operators down the spine of a session type by applying the conversion rules C-DualIn, C-DualOut, C-DualEnd?, and C-DualEnd! from left to right. If it encounters another `Dual` operator along the spine, it applies rule C-DualInv. We cannot further resolve `Dual` α, hence it is a normal form.

The reverse operator only applies to protocol types. So it can only occur in a message type like $?(-T).S$ and in the parameter of a protocol type. In a message type, normalization must apply rules C-NegIn and C-NegOut exhaustively from left to right, which removes the reverse operator from message types. In the parameter of a protocol type, normalization applies rule C-NegInv exhaustively. Starting with an odd number of reverse operators, a single operator remains; starting with an even number, all reverse operators are removed.

Following this intuition normalization is defined in Figure 3 by two mutually recursive functions $\mathrm{nrm}^+(\_)$ and $\mathrm{nrm}^-(\_)$ along with three auxiliary functions. Positive normalization $\mathrm{nrm}^+(\_)$ traverses and reconstructs all non-session type constructs. It also gets invoked on the type of the transmitted value for session types, i.e., on T in type $!T.S$. Normalizing the `Dual` operator switches forth and back between positive and negative normalization, $\mathrm{nrm}^+(\_)$ and $\mathrm{nrm}^-(\_)$.

Negative normalization with superscript $^-$ indicates a pending `Dual` type constructor, so that the function $\mathrm{nrm}^-(\_)$ is only used for session types. Normalization pushes the pending dual constructor along the spine of a session type to implement rules C-DualIn, C-DualOut, C-DualEnd?, and C-DualEnd!. The pending dualization gets reified on a type variable.

Normalizing a reversed protocol $-T$ invokes the negative directional function $-(T')$ on T's normal form to enforce the conversion rule C-DualInv.

Type normalization $\boxed{\mathrm{nrm}^+(T) = T \quad \mathrm{nrm}^-(T) = T}$

$$\begin{aligned}
\mathrm{nrm}^+(\texttt{Unit}) &= \texttt{Unit} & \mathrm{nrm}^+(\texttt{Dual}\ T) &= \mathrm{nrm}^-(T)\\
\mathrm{nrm}^+(T \to U) &= \mathrm{nrm}^+(T) \to \mathrm{nrm}^+(U) & \mathrm{nrm}^+(\rho\ \overline{T}) &= \rho\ \overline{\mathrm{nrm}^+(T)}\\
\mathrm{nrm}^+(T \otimes U) &= \mathrm{nrm}^+(T) \otimes \mathrm{nrm}^+(U) & \mathrm{nrm}^+(-T) &= -(\mathrm{nrm}^+(T))\\
\mathrm{nrm}^+(\forall\alpha\colon\kappa.T) &= \forall\alpha\colon\kappa.\mathrm{nrm}^+(T) & \mathrm{nrm}^-(\texttt{Dual}\ T) &= \mathrm{nrm}^+(T)\\
\mathrm{nrm}^+(\alpha) &= \alpha & \mathrm{nrm}^-(\alpha) &= \texttt{Dual}\ \alpha\\
\mathrm{nrm}^+(?T.S) &= \S(-(\mathrm{nrm}^+(T))).\mathrm{nrm}^+(S) & \mathrm{nrm}^-(?T.S) &= \S(+(\mathrm{nrm}^+(T))).\mathrm{nrm}^-(S)\\
\mathrm{nrm}^+(!T.S) &= \S(+(\mathrm{nrm}^+(T))).\mathrm{nrm}^+(S) & \mathrm{nrm}^-(!T.S) &= \S(-(\mathrm{nrm}^+(T))).\mathrm{nrm}^-(S)\\
\mathrm{nrm}^+(\texttt{End}_?) &= \texttt{End}_? & \mathrm{nrm}^-(\texttt{End}_?) &= \texttt{End}_!\\
\mathrm{nrm}^+(\texttt{End}_!) &= \texttt{End}_! & \mathrm{nrm}^-(\texttt{End}_!) &= \texttt{End}_?
\end{aligned}$$

Materialization and the directional operators $\boxed{\S(T).T = T \quad -(T) = T \quad +(T) = T}$

$$\begin{aligned}
\S(-T).U &= ?T.U & -(-T) &= +(T) & +(-T) &= -(T)\\
\S(T).U &= !T.U & -(T) &= -T & +(T) &= T
\end{aligned}$$

In the second line $T \neq -U$ in all cases.

Fig. 3. Type normalization and auxiliary metafunctions on session types and protocol types

Here is an example of a step-by-step computation of the positive normal form of $\texttt{Dual}\ (?(-\texttt{Int}).\alpha)$.

$$\begin{aligned}
&\mathrm{nrm}^+(\texttt{Dual}\ (?(-\texttt{Int}).\alpha))\\
&= \mathrm{nrm}^-(?(-\texttt{Int}).\alpha) && \text{negative normalization remembers pending } \texttt{Dual}\\
&= \S(+(\mathrm{nrm}^+(-\texttt{Int}))).\mathrm{nrm}^-(\alpha) && \mathrm{nrm}^+(_) \text{ on the payload, } \mathrm{nrm}^-(_) \text{ on the spine}\\
&= \S(+(-(\mathrm{nrm}^+(\texttt{Int})))).\texttt{Dual}\ \alpha && \text{negative normalization reifies } \texttt{Dual} \text{ on type variable}\\
&= \S(-\texttt{Int}).\texttt{Dual}\ \alpha && \text{materialization applied to normal form } \ldots\\
&= ?\texttt{Int}.\texttt{Dual}\ \alpha && \ldots \text{ fixes the direction}
\end{aligned}$$

To test whether a type equivalence goal $\Delta \vdash T \equiv U : \kappa$ holds, we compute *algorithmic type equivalence*, notation $T \equiv_A U$, which is defined by comparision of normal forms: $\mathrm{nrm}^+(T) = \mathrm{nrm}^+(U)$ up to α-equivalence. The following propositions establish soundness and completeness of the normalization algorithm, as well as its worst case running time.

Theorem 3.1 (Soundness). *Let* $\Delta \vdash T \Rightarrow \kappa$ *and* $\Delta \vdash U \Rightarrow \kappa$.

(1) *If* $\mathrm{nrm}^+(T) =_\alpha \mathrm{nrm}^+(U)$, *then* $\Delta \vdash T \equiv U : \kappa$.
(2) *If* $\kappa = \boldsymbol{s}$ *and* $\mathrm{nrm}^-(T) =_\alpha \mathrm{nrm}^-(U)$, *then* $\Delta \vdash T \equiv U : \boldsymbol{s}$.

Theorem 3.2 (Completeness). *Let* $\Delta \vdash T \equiv U : \kappa$.

(1) $\mathrm{nrm}^+(T) =_\alpha \mathrm{nrm}^+(U)$.
(2) *If* $\kappa = \boldsymbol{s}$, *then* $\mathrm{nrm}^-(T) =_\alpha \mathrm{nrm}^-(U)$.

Theorem 3.3 (Time complexity of type equivalence). *The test for* $T \equiv_A U$ *runs in* $O(|T| + |U|)$.

Types for constants $\boxed{\mathsf{typeof}(c) = T}$

$$\mathsf{typeof}(*) = \mathtt{Unit} \qquad \mathsf{typeof}(\mathtt{fork}) = (\mathtt{Unit} \to \mathtt{Unit}) \to \mathtt{Unit}$$
$$\mathsf{typeof}(\mathtt{new}) = \forall\alpha\colon \mathbf{s}.\alpha \otimes \mathtt{Dual}\ \alpha \qquad \mathsf{typeof}(\mathtt{terminate}) = \mathtt{End}_! \to \mathtt{Unit}$$
$$\mathsf{typeof}(\mathtt{receive}) = \forall\alpha\colon \mathbf{T}.\forall\beta\colon \mathbf{s}.?\alpha.\beta \to \alpha \otimes \beta \qquad \mathsf{typeof}(\mathtt{wait}) = \mathtt{End}_? \to \mathtt{Unit}$$
$$\mathsf{typeof}(\mathtt{send}) = \forall\alpha\colon \mathbf{T}.\forall\beta\colon \mathbf{s}.\alpha \to !\alpha.\beta \to \beta$$
$$\mathsf{typeof}(\mathtt{select}\ C_k) = \forall\overline{\alpha}\colon \overline{\mathbf{P}}.\forall\beta\colon \mathbf{s}.!(\rho\ \overline{\alpha}).\beta \to \S(+(\overline{T_k})).\beta \text{ if } \mathtt{protocol}\ \rho\ \overline{\alpha} = \{C_i\ \overline{T_i}\}_{i\in I} \text{ and } k \in I$$

Fig. 4. Types for constants

4 Expressions and Processes

This section introduces the notions of expressions and process configurations, algorithmic typing for expressions, and operational semantics for expressions and configurations based on labelled transition relations. The syntax of constants c, values v, expressions e, and process configurations K is defined by the grammar below.

$$\begin{aligned}
c &::= * \mid \mathtt{fork} \mid \mathtt{new} \mid \mathtt{receive} \mid \mathtt{send} \mid \mathtt{select}\,C \mid \mathtt{wait} \mid \mathtt{terminate} \\
v &::= c \mid x \mid \lambda x\colon T.e \mid \mathtt{rec}\ x\colon T.v \mid \Lambda\alpha\colon\kappa.v \mid \langle v, v\rangle \mid \mathtt{rec}\ x\colon T.v \mid \mathtt{receive}[T] \\
&\quad \mathtt{receive}[T][T] \mid \mathtt{send}[T] \mid \mathtt{send}[T][T] \mid \mathtt{send}[T][T]\,v \mid \mathtt{select}\,C[\overline{T}] \\
e &::= v \mid e\ e \mid e[T] \mid \mathtt{let}\ * = e\ \mathtt{in}\ e \mid \langle e, e\rangle \mid \mathtt{let}\ \langle x, x\rangle = e\ \mathtt{in}\ e \mid \\
&\quad \mathtt{match}\ e\ \mathtt{with}\ \{C_i\ x_i \to e_i\}_{i\in I} \\
E &::= \{e_1, \ldots, e_m\} \\
K &::= (E, L)
\end{aligned}$$

All constructors are standard either in linear functional languages or in session type languages. From functional languages we find function introduction and elimination ($\lambda x\colon T.e$ and $e_1\ e_2$), linear pair introduction and elimination ($\langle e_1, e_2\rangle$ and $\mathtt{let}\ \langle x, y\rangle = e_1\ \mathtt{in}\ e_2$), linear unit introduction and elimination ($*$ and $\mathtt{let}\ * = e_1\ \mathtt{in}\ e_2$). From session languages we find most of the constructors in the form of constants, including `fork` to spawn a new thread, `new` to create a new channel, `receive` to receive a value on a given channel, `send` to send a value on a given channel, `wait` to wait for a channel to be closed, `terminate` to force channel closing, and $\mathtt{select}\,C$ to exercise a choice on a given channel indicated by selector name C. The operator to offer a choice on some channel cannot be captured by a type in our type language, hence we have made it an expression ($\mathtt{match}\ e\ \mathtt{with}\ \{C_i\ x_i \to e_i\}_{i\in I}$). At the level of processes, a configuration $K = (E, L)$ consists of a finite multiset E of expressions and a finite set L of live channels. Each element $\{x, y\} \in L$ is an unordered pair of distinct variables naming the two ends of one channel. The expressions are closed with respect to type variables and may share the channel-end variables recorded by L. The multiset E is the pool of parallel threads; multiplicity distinguishes separate occurrences of the same expression, while order is immaterial. Closing a channel removes its pair from L, while allocation adds a pair of fresh variables. This flat representation avoids structural congruence and scope-extrusion rules.

Expression typing is again mostly standard with respect to (linear) functional or session type languages. The definitions for typing assume a set of protocol declarations as an implicit argument. The typings of $\mathtt{select}\,C$ (Figure 4) and `match` (Figure 5, E-Match) access these protocol declarations. The typeof operator in Figure 4 returns the types for constants in normal form. The typing rules for expressions are defined in Figure 5. The type system is algorithmic, implemented with two

Type synthesis $\boxed{\Delta \mid \Gamma \vdash e \Rightarrow T \mid \Gamma}$

$$\textsc{E-Const}\quad \Delta \mid \Gamma \vdash c \Rightarrow \mathrm{typeof}(c) \mid \Gamma$$

$$\textsc{E-Var}\quad \frac{\Delta \vdash T \Leftarrow \mathbf{T}}{\Delta \mid \Gamma, x\colon T \vdash x \Rightarrow T \mid \Gamma} \qquad \textsc{E-Var}\star\quad \frac{\Delta \vdash T \Leftarrow \mathbf{T}}{\Delta \mid \Gamma, x\colon^{\star} T \vdash x \Rightarrow T \mid \Gamma, x\colon^{\star} T}$$

$$\textsc{E-Let}^{*}\quad \frac{\Delta \mid \Gamma_1 \vdash e_1 \Leftarrow \mathtt{Unit} \mid \Gamma_2 \qquad \Delta \mid \Gamma_2 \vdash e_2 \Rightarrow T \mid \Gamma_3}{\Delta \mid \Gamma_1 \vdash \mathtt{let} * = e_1 \,\mathtt{in}\, e_2 \Rightarrow T \mid \Gamma_3}$$

$$\textsc{E-Abs}\quad \frac{\Delta \vdash T \Leftarrow \mathbf{T} \qquad \mathrm{nrm}^{+}(T) = V \qquad \Delta \mid \Gamma_1, x\colon V \vdash e \Rightarrow U \mid \Gamma_2 \qquad x \notin \Gamma_2}{\Delta \mid \Gamma_1 \vdash \lambda x\colon T.e \Rightarrow V \to U \mid \Gamma_2}$$

$$\textsc{E-App}\quad \frac{\Delta \mid \Gamma_1 \vdash e_1 \Rightarrow T \to U \mid \Gamma_2 \qquad \Delta \mid \Gamma_2 \vdash e_2 \Leftarrow T \mid \Gamma_3}{\Delta \mid \Gamma_1 \vdash e_1\, e_2 \Rightarrow U \mid \Gamma_3} \qquad \textsc{E-Pair}\quad \frac{\Delta \mid \Gamma_1 \vdash e_1 \Rightarrow T \mid \Gamma_2 \qquad \Delta \mid \Gamma_2 \vdash e_2 \Rightarrow U \mid \Gamma_3}{\Delta \mid \Gamma_1 \vdash \langle e_1, e_2 \rangle \Rightarrow T \otimes U \mid \Gamma_3}$$

$$\textsc{E-Rec}\quad \frac{\Delta \vdash T \Leftarrow \mathbf{T} \qquad \mathrm{nrm}^{+}(T \to U) = V \qquad \Delta \mid \Gamma, x\colon^{\star} V \vdash v \Leftarrow V \mid \Gamma, x\colon^{\star} V}{\Delta \mid \Gamma \vdash \mathtt{rec}\, x\colon T \to U.v \Rightarrow V \mid \Gamma}$$

$$\textsc{E-TAbs}\quad \frac{\Delta, \alpha\colon \kappa \mid \Gamma_1 \vdash v \Rightarrow T \mid \Gamma_2}{\Delta \mid \Gamma_1 \vdash \Lambda \alpha\colon \kappa.v \Rightarrow \forall \alpha\colon \kappa.T \mid \Gamma_2} \qquad \textsc{E-TApp}\quad \frac{\Delta \mid \Gamma_1 \vdash e \Rightarrow \forall \alpha\colon \kappa.U \mid \Gamma_2 \qquad \Delta \vdash T \Leftarrow \kappa}{\Delta \mid \Gamma_1 \vdash e[T] \Rightarrow \mathrm{nrm}^{+}(U[T/\alpha]) \mid \Gamma_2}$$

$$\textsc{E-Let}\quad \frac{\Delta \mid \Gamma_1 \vdash e_1 \Rightarrow T \otimes U \mid \Gamma_2 \qquad \Delta \mid \Gamma_2, x\colon \mathrm{nrm}^{+}(T), y\colon \mathrm{nrm}^{+}(U) \vdash e_2 \Rightarrow V \mid \Gamma_3 \qquad x, y \notin \Gamma_3}{\Delta \mid \Gamma_1 \vdash \mathtt{let}\, \langle x, y \rangle = e_1 \,\mathtt{in}\, e_2 \Rightarrow V \mid \Gamma_3}$$

$$\textsc{E-Match}\quad \frac{\mathtt{protocol}\ \rho\ \overline{\alpha} = \{C_i\ \overline{T_i}\}_{i \in I} \qquad \forall i \in I:\ \Delta \mid \Gamma_2, x_i\colon \S(-(\overline{T_i}[\overline{U}/\overline{\alpha}])).S \vdash e_i \Rightarrow V_i \mid \Gamma_i \qquad \Delta \mid \Gamma_1 \vdash e \Rightarrow ?(\rho\ \overline{U}).S \mid \Gamma_2 \qquad x_i \notin \Gamma_i \qquad \Gamma_3 =_{\alpha} \Gamma_i \qquad V = \bigsqcup_{i \in I} V_i}{\Delta \mid \Gamma_1 \vdash \mathtt{match}\ e\ \mathtt{with}\ \{C_i\ x_i \to e_i\}_{i \in I} \Rightarrow V \mid \Gamma_3}$$

Type against $\boxed{\Delta \mid \Gamma \vdash e \Leftarrow T \mid \Gamma}$

$$\textsc{E-Check}\quad \frac{\Delta \mid \Gamma_1 \vdash e \Rightarrow U \mid \Gamma_2 \qquad U \leq T}{\Delta \mid \Gamma_1 \vdash e \Leftarrow T \mid \Gamma_2}$$

Fig. 5. Algorithmic typing rules for expressions

mutually recursive judgments: one to synthetize a type of an expression, the other to check an expression against a type. The typing rules rely on type variable contexts Δ introduced in Section 3, on term variable contexts Γ assigning types T to term variables x. There are two sorts of entries in term variable contexts: linear entries, noted $x\colon T$, and unrestricted entries, noted $x\colon^{\star} T$. The latter are used for recursive functions only (cf. rules E-Var$\star$ and E-Rec) and are inspired by work on quantitative type theory [6, 12, 36].

The type system maintains the invariant that entries in term variable contexts are always normalized: the rules that write into contexts, namely E-Abs, E-Rec and E-Let, normalize the new

entry. Judgments are of the form $\Delta \mid \Gamma_1 \vdash e \Rightarrow T \mid \Gamma_2$ or $\Delta \mid \Gamma_1 \vdash e \Leftarrow T \mid \Gamma_2$, where Γ_2 represents the part of context Γ_1 [57] that is not consumed by e and T is in normal form. The axioms (E-Const and E-Var) mostly copy the input context to the output. Rule E-App is a good example of context threading: to type expression $e_1\ e_2$, expression e_1 is given the input context Γ_1, produces Γ_2 which is given to e_2, which turn produces Γ_3 that constitutes the outgoing context of application $e_1\ e_2$. The rules that add linear entries to the context (E-Abs, E-Let and E-Match) all check that the new variables are not present in the outgoing context, thus ensuring that the corresponding values are fully consumed. Rule E-TApp explicitly normalizes the output type for normalization is not preserved by substitution; all rules that introduce new entries in the context (E-Abs, E-Rec, E-Let, and E-Match) make sure that types are normalized, thus ensuring that contexts contain only normalized types.

Rules E-Rec and E-Var$\star$ govern the typing of recursive functions. Rule E-Rec binds the recursive function x using an unrestricted binding $x{:}^{\star}\_$ so that any number of recursive calls is possible in the body via E-Var$\star$. Unrestricted bindings provide for termination of recursive functions. Such functions cannot use linear variables from the environment, which we enforce by requiring the incoming context Γ to be equal to the outgoing context.

The truly new rule is E-Match, which indicates that all rules are parametric with respect to an implicit set of protocol declarations: there is a declaration for each protocol ρ which has a kinding in Δ. The rule starts by synthesizing the input type of the expression to be matched; the type is $?(\rho\ \overline{U}).S$ for some protocol constructor ρ. At this point, the declaration of protocol ρ is looked up in the implicit set of declarations. The `match` branch for each choice of selector C_i is then typed in turn. The parameters $\overline{\alpha}$ of the protocol are instantiated with the arguments $\overline{U}$ to produce a sequence of types $\overline{T_i}[\overline{U}/\overline{\alpha}]$. This sequence of types is converted to a normalized session type as described in Section 3, which is assigned to variable x_i. Now, each branch synthesizes a type V_i and context Γ_i, for $i \in I$. [MW] : In all branches, the contexts must coincide (up to α equivalence) with the output context of the `match`. The output type V is the least upper bound of the branch types.

In this rule, and also in the type for `select` in Figure 4, we extend the directional operators and materialization (from Figure 3) to sequences of parameters by mapping: $\pm(\overline{T}) = \overline{\pm(T)}$ as well as $\S(\varepsilon).S = S$ and $\S(T\overline{T}).S = \S(T).\S(\overline{T}).S$.

[MW] : Rule E-Check relies on the previous judgement to check expression e against type T, up to subtyping. Both types can be assumed to have normal form, because synthesis outputs U in normal form and checking must be invoked with T in normal form.

The operational semantics for expressions and process configurations is defined via transition relations labelled by the actions below.

$$
\begin{array}{lcll}
\sigma & ::= & x?v \mid x!v \mid x?C \mid x!C \mid x? & \text{labels for session operations} \\
\lambda & ::= & \sigma \mid \beta \mid \mathtt{fork}\, v \mid (\nu xy)\, T & \text{labels for expressions} \\
\pi & ::= & \tau \mid (\nu xy)\, T & \text{labels for processes}
\end{array}
$$

The σ labels capture value and selector communication and the close-style action on a session endpoint. The mechanization uses one close label for both ends of a closing synchronization. Labels λ are for expressions: β is for internal actions, $\mathtt{fork}\, v$ records a forked value, and $(\nu xy)\, T$ records the type and the two fresh endpoint names of an allocated channel. Process labels retain only the information visible at the configuration level: τ is an internal step and $(\nu xy)\, T$ announces allocation of the fresh channel $\{x, y\}$. Session actions do not escape as process labels; they either synchronize two distinct threads or leave the thread waiting for a partner.

The axioms of the labelled transition system for expressions is defined by the rules in Figure 6 (Section D, Figure 12, contains the whole set). We group under label β the common reduction rules

Labelled transition system for expressions $\boxed{e \xrightarrow{\lambda} e}$

Act-App
$$(\lambda x\colon T.e)\, v \xrightarrow{\beta} e[v/x]$$

Act-TApp
$$(\Lambda\, \alpha\colon \kappa.v)[T] \xrightarrow{\beta} v[T/\alpha]$$

Act-Let
$$\mathtt{let}\ \langle x, y\rangle = \langle u, v\rangle\ \mathtt{in}\ e \xrightarrow{\beta} e[u/x][v/y]$$

Act-Let*
$$\mathtt{let} * = *\ \mathtt{in}\ e \xrightarrow{\beta} e$$

Act-Rec
$$(\mathtt{rec}\ x\colon T.v)\, u \xrightarrow{\beta} (v[\mathtt{rec}\ x\colon T.v/x])\, u$$

Act-Fork
$$\mathtt{fork}\, v \xrightarrow{\mathtt{fork}\, v} *$$

Act-New
$$\mathtt{new}[T] \xrightarrow{(vxy)\, T} \langle x, y\rangle$$

Act-Rcv
$$\mathtt{receive}[T][U]\, x \xrightarrow{x?v} \langle v, x\rangle$$

Act-Send
$$\mathtt{send}[T][U]\, v\, x \xrightarrow{x!v} x$$

Act-Match
$$\mathtt{match}\ x\ \mathtt{with}\ \{C_i\ y_i \to e_i\}_{i\in I} \xrightarrow{x?C_k} e_k[x/y_k]$$

Act-Sel
$$\mathtt{select}\, C[\overline{T}]\, x \xrightarrow{x!C} x$$

Act-Wait
$$\mathtt{wait}\, x \xrightarrow{x?} *$$

Act-Term
$$\mathtt{terminate}\, x \xrightarrow{x?} *$$

Fig. 6. Labelled transition system for expressions (selected rules). The full version, including call-by-value structural rules, is presented in Section D, Figure 12

Labelled transition system for process configurations $\boxed{K \xrightarrow{\pi} K'}$

Act-Beta
$$\frac{e \xrightarrow{\beta} e'}{(\{e\} \uplus E, L) \xrightarrow{\tau} (\{e'\} \uplus E, L)}$$

Act-Fork
$$\frac{e \xrightarrow{\mathtt{fork}\, v} e'}{(\{e\} \uplus E, L) \xrightarrow{\tau} (\{e', v\, *\} \uplus E, L)}$$

Act-New
$$\frac{e \xrightarrow{(vxy)\, S} e' \qquad x \neq y \qquad x, y \notin (\{e\} \uplus E, L)}{(\{e\} \uplus E, L) \xrightarrow{(vxy)\, S} (\{e'\} \uplus E, L \cup \{\{x, y\}\})}$$

Act-Msg
$$\frac{\{x, y\} \in L \qquad e_1 \xrightarrow{x?v} e_1' \qquad e_2 \xrightarrow{y!v} e_2'}{(\{e_1, e_2\} \uplus E, L) \xrightarrow{\tau} (\{e_1', e_2'\} \uplus E, L)}$$

Act-Bra
$$\frac{\{x, y\} \in L \qquad e_1 \xrightarrow{x?C} e_1' \qquad e_2 \xrightarrow{y!C} e_2'}{(\{e_1, e_2\} \uplus E, L) \xrightarrow{\tau} (\{e_1', e_2'\} \uplus E, L)}$$

Act-Wait
$$\frac{\{x, y\} \in L \qquad e_1 \xrightarrow{x?} e_1' \qquad e_2 \xrightarrow{y?} e_2'}{(\{e_1, e_2\} \uplus E, L) \xrightarrow{\tau} (\{e_1', e_2'\} \uplus E, L \setminus \{\{x, y\}\})}$$

Fig. 7. Labelled transition system for process configurations

of the polymorphic lambda calculus. Rules Act-Fork and Act-New record, respectively, the value forked and the type and endpoint names of the freshly allocated channel. These transitions are then handled at the configuration level. The remaining observable labels cover value communication, selector communication, and closing.

The labelled transition system for configurations is defined by the rules in Figure 7. Multiset union is written $\uplus$. A source of the form $(\{e\} \uplus E, L)$ distinguishes one occurrence of e and leaves the remainder E unchanged; similarly, $(\{e_1, e_2\} \uplus E, L)$ distinguishes two thread occurrences. Thus the rules require neither thread positions nor an explicit premise that the two occurrences are

Liveness/context agreement $\boxed{\mathrm{LiveCtx}(L;\Gamma)}$

$$\mathrm{LiveCtx}(L;\Gamma) \iff \mathrm{ends}(L) = \mathrm{dom}(\Gamma), \qquad \mathrm{ends}(L) = \{x \mid \exists y.\ \{x,y\} \in L\}.$$

Splitting a configuration context $\boxed{\Gamma = \Gamma_1 + \Gamma_2}$

$$\textsc{S-Empty}\ \ \cdot = \cdot + \cdot \qquad \textsc{S-LinL}\ \ \frac{\Gamma = \Gamma_1 + \Gamma_2}{\Gamma, x\colon S = (\Gamma_1, x\colon S) + \Gamma_2} \qquad \textsc{S-LinR}\ \ \frac{\Gamma = \Gamma_1 + \Gamma_2}{\Gamma, x\colon S = \Gamma_1 + (\Gamma_2, x\colon S)}$$

Typing a thread multiset $\boxed{\Gamma \vdash_{\mathsf{th}} E}$

$$\textsc{TM-Empty}\ \ \cdot \vdash_{\mathsf{th}} \emptyset \qquad \textsc{TM-Thread}\ \ \frac{\Gamma = \Gamma_e + \Gamma_r \qquad \cdot \mid \Gamma_e \vdash e \Leftarrow \mathtt{Unit} \mid \cdot \qquad \Gamma_r \vdash_{\mathsf{th}} E}{\Gamma \vdash_{\mathsf{th}} \{e\} \uplus E}$$

Typing a process configuration $\boxed{\Gamma \vdash K}$

$$\textsc{T-Conf}\ \ \frac{\mathrm{LiveCtx}(L;\Gamma) \qquad \Gamma \vdash_{\mathsf{th}} E}{\Gamma \vdash (E, L)}$$

Fig. 8. Typing rules for process configurations. Context addition is disjoint union, and $\mathrm{dom}(\Gamma)$ is the set of variables bound in Γ.

distinct. The notation $x \notin K$ means that x occurs neither in an expression of K nor in a pair in its set of live channels.

Rules Act-Beta and Act-Fork lift independent expression transitions for an arbitrary thread; the latter adds the application $v\ *$ as a new thread. Rule Act-New chooses distinct names x and y that are fresh for the entire source configuration, exposes them in the allocation label, and adds $\{x, y\}$ to the set of live channels. The remaining rules synchronize two thread occurrences whose endpoints form a pair in L. They replace both occurrences in the multiset, and Act-Wait additionally removes the synchronized pair from L. Their synchronization discipline follows prior labelled semantics for linear functional languages with sessions [19, 20, 37]. In the flat presentation, parallel composition, restriction, scope opening, and their associated structural rules are absorbed by multiset decomposition and the explicit set of live channels. The mechanization gives the full formal details of a concrete representation and its permutation invariance.

We complete the section with the main results of our language. Towards this end we need to talk about context and configuration typing. Type formation (Figure 1) is lifted pointwise to contexts in judgement $\Delta \vdash \Gamma \Leftarrow \kappa$. The rules for typing process configurations are in Figure 8. A configuration context records named channel endpoints and their linear session types. Thus $\mathrm{LiveCtx}(L;\Gamma)$ states that the domain of Γ is exactly the set of endpoints occurring in the pairs of L. Endpoints of closed channels do not occur in the context. The thread-multiset judgment selects an expression e from a decomposition $\{e\} \uplus E$ and partitions the context as $\Gamma = \Gamma_e + \Gamma_r$. Context addition is disjoint, so each binding is assigned either to e or to the remainder E. The selected expression must check against $\mathtt{Unit}$ and consume its entire context, leaving the empty context. Correspondingly, the empty thread multiset is typable only under the empty context. Consequently every live endpoint is owned and consumed by exactly one thread.

Expression preservation still uses the labelled context transition from Section D (Figure 13). At the configuration level the updated context is instead constructed together with the target

typing derivation. A raw synchronization step records live peer endpoints, but not the coherence of their session types. We write redTyped$(\mathcal{D}, s)$ when a step s is compatible with a source typing derivation $\mathcal{D}$; this condition is immediate for beta reduction, fork, and allocation, and supplies the endpoint and context-effect compatibility for the three synchronization rules. This is the paper-level presentation of the mechanized ReductionTyping judgment.

Theorem 4.1 (Preservation).

(1) *If* $e_1 \xrightarrow{\lambda} e_2$ *and* $\Delta \mid \Gamma_1, \Gamma_2, \Gamma_3 \vdash e_1 \Rightarrow T \mid \Gamma_3$ *and* $\Gamma_2 \xrightarrow{\lambda} \Gamma_2'$ *and* $\Delta \vdash \Gamma_1, \Gamma_2, \Gamma_3 \Leftarrow \mathbf{T}$, *then* $\Delta \mid \Gamma_1, \Gamma_2', \Gamma_3 \vdash e_2 \Rightarrow T' \mid \Gamma_3$ *and* $T' \leq T$.

(2) *If* $e_1 \xrightarrow{\lambda} e_2$ *and* $\Delta \mid \Gamma_1, \Gamma_2, \Gamma_3 \vdash e_1 \Leftarrow T \mid \Gamma_3$ *and* $\Gamma_2 \xrightarrow{\lambda} \Gamma_2'$ *and* $\Delta \vdash \Gamma_1, \Gamma_2, \Gamma_3 \Leftarrow \mathbf{T}$, *then* $\Delta \mid \Gamma_1, \Gamma_2', \Gamma_3 \vdash e_2 \Leftarrow T \mid \Gamma_3$.

(3) *If* $\mathcal{D}$ *derives* $\Gamma \vdash K$, *where* $s = K \xrightarrow{\pi} K'$, *and* redTyped$(\mathcal{D}, s)$, *then there exists a context* Γ' *such that* $\Gamma' \vdash K'$.

For progress, expression actions are divided according to whether they need a second thread. An expression is *runnable* if it can take a β, fork, or allocation step. It is *communication-blocked* if it has a value-receive, value-send, selector-receive, selector-send, or close transition; such an expression is not stuck locally, but its transition needs a matching thread. We write RunnableAt(K) when some thread is runnable and Sync(K) when two distinct thread occurrences satisfy the premises of Act-Msg, Act-Bra, or Act-Wait.

A configuration is *terminal* if every thread is a value. A *global deadlock* is positive evidence consisting of all four properties below:

(1) every thread is either a value or communication-blocked;
(2) at least one thread is communication-blocked;
(3) RunnableAt(K) does not hold; and
(4) Sync(K) does not hold.

The last two clauses exclude every source of a configuration transition, so a global deadlock cannot step.

The mechanization proves the global lifting below. The local expression progress result and the finite decision procedures are explicit hypotheses; their full statements are recorded in the mechanization.

Theorem 4.2 (Configuration progress). *Assume local expression progress: every expression that checks against* `Unit` *in a session-only context is a value, runnable, or communication-blocked. Also assume that* RunnableAt(K) *and* Sync(K) *are decidable. If* $\Gamma \vdash K$, *then either* K *is terminal, it is a global deadlock, or there exist* π *and* K' *such that* $K \xrightarrow{\pi} K'$.

5 Modelling Recursive Session Types via Algebraic Protocols

This section describes how recursive session types à la Gay and Vasconcelos [24] can be embedded in algebraic protocols. To differentiate the two different type languages we use "algebraic types" to refer to session and protocol types as defined by Mordido et al. [38], "recursive types" refer to classical, end-recursive session types. Both notions also include the corresponding functional types.

5.1 A graph representation for recursive types

Types are represented via deterministic labelled transition systems $(V, L, \longrightarrow)$ where V is a set of nodes or states, L is the set of transition labels and $\longrightarrow \subseteq V \times L \times V$ the transition relation. We may write $v \xrightarrow{\ell} v'$ for $(v, \ell, v') \in \longrightarrow$, and $v \nrightarrow$ if there are no ℓ, v' such that $v \xrightarrow{\ell} v'$.

Given a set of base types B, and a set of branch and choice labels C the transition labels L comprise[5]

$$\begin{aligned} L &= \{!, ?, \&, +, \mathsf{End}, \rightarrow, 1, 2\} \uplus B \uplus C \\ L_{Con} &= \{!, ?, \&, +, \mathsf{End}, \rightarrow\} \uplus B && \subseteq L \\ L^S_{Con} &= \{!, ?, \&, +, \mathsf{End}\} && \subseteq L_{Con} \end{aligned}$$

The sets L_{Con} and L^S_{Con} give names to specific subsets of L. The former denotes the set of transition labels that correspond to type constructors. The latter corresponds to the subset corresponding to session type constructors.

There needs to be a way to associate the top-level type constructor that corresponds with each node in way that is respected by bisimilarity. To this end we require one designated node $o \in V$ with $o \not\rightarrow$. From every node $v \in V, v \neq o$ there must be exactly one transition to o. That transition must be labelled from the set L_{Con}. The function $\mathscr{L}(v)$ returns the label ℓ from the transition $v \xrightarrow{\ell} o$. Besides a singular transition to o, for every nodes $v \in V, v \neq o$ the conditions below must be satisfied.

(1) If $\mathscr{L}(v) = \mathsf{End}$ or $\mathscr{L}(v) \in B$ it must hold that $v \not\rightarrow$.
(2) If $\mathscr{L}(v) \in \{!, ?, \rightarrow\}$ there must be $v_1, v_2 \in V$ with $v \xrightarrow{1} v_1$ and $v \xrightarrow{2} v_2$, and for all $v \xrightarrow{\ell} v'$ it must hold that $(\ell, v') \in \{(1, v_1), (2, v_2), (\rightarrow, o)\}$.
(3) If $\mathscr{L}(v) \in \{+, \&\}$ there must be at least one transition $v \xrightarrow{\ell} v'$ with $\ell \in C$, and for all transitions $v \xrightarrow{\ell'} v_\ell$ it must hold that $v_\ell \neq o$ iff $\ell' \in C$.
(4) If $\mathscr{L}(v) \in L^S_{Con}$, for all $v \xrightarrow{\ell} v'$ with $\ell \neq 1$ and $v' \neq o$ it must hold that $\mathscr{L}(v') \in L^S_{Con}$.
(5) If there is a cycle $v \xrightarrow{\ell} v_1 \xrightarrow{\ell_1} \cdots v_n \xrightarrow{\ell_n} v_1$ it must hold that $\mathscr{L}(v) \in L^S_{Con}$ and $\ell \neq 1$.
(6) If $v \xrightarrow{\ell} v'$ and $v \xrightarrow{\ell} v''$ it must hold that $v' = v''$.

Requirement (5) restricts the kind of cycles that may occur. All cycles must consists solely of session type nodes and the transition labels must not be 1. The restriction serves two goals: only sessions types are allowed to be recursive and we avoid the dualization paradox where recursive session types use recursion variables in the payload of messages [11, 25].

Definition 5.1. The functions post, cycles, non-cycles : $V \rightarrow L \times V$ return the set of outgoing transitions and their targets—excluding transitions to o. cycles is restricted to transitions that are part of a cycle. non-cycles is its complement with respect to post.

$$\begin{aligned} \mathrm{post}(v) &= \left\{(\ell, v') \in L \times V \,\middle|\, v \xrightarrow{\ell} v',\ v' \neq o\right\} \\ \mathrm{cycles}(v) &= \left\{(\ell, v') \in \mathrm{post}(v) \,\middle|\, \exists n \geq 0, v_1 \ldots v_n, \ell_1 \ldots \ell_{n+1}. v' \xrightarrow{\ell_1} v_1 \xrightarrow{\ell_2} \cdots v_n \xrightarrow{\ell_{n+1}} v\right\} \\ \text{non-cycles}(v) &= \mathrm{post}(v) \setminus \mathrm{cycles}(v) \end{aligned}$$

Given an LTS $A = (V, L, \longrightarrow)$ the function $\mathcal{T}_A$ produces a type with the grammar below. In particular, if $\mathscr{L}(v) \in L^S_{Con}$ for some $v \in V$, the result of $\mathcal{T}_A(v, \eta)$ is a type according to the grammar S. If the LTS parameter is obvious from context we may omit the subscript and write $\mathcal{T}(v, \eta)$ instead.

$$\begin{aligned} T &::= \ \ldots \\ S &::= \ \ldots \end{aligned}$$

[5]The operator $\cdot \uplus \cdot$ denotes disjoint union, i. e. we assume the three sets to be distinct.

$$\mathcal{T}(v,\eta) = \begin{cases} v & \text{if } v \in \eta \\ \mu v.\star\langle \ell : \mathcal{T}(v', \eta \cup \{v\})\rangle_{(\ell,v')\in I} & \text{if } v \notin \eta,\ \star = \mathscr{L}(v) \in \{\&, +\},\ I = \mathrm{post}(v) \\ \mu v.\sharp\mathcal{T}(v_1,\emptyset).\mathcal{T}(v_2, \eta \cup \{v\}) & \text{if } v \notin \eta,\ \sharp = \mathscr{L}(v) \in \{!, ?\},\ v \xrightarrow{1} v_1, v \xrightarrow{2} v_2 \\ \mathcal{T}(v_1,\emptyset) \to \mathcal{T}(v_2,\emptyset) & \text{if } \mathscr{L}(v) = \to,\ v \xrightarrow{1} v_1, v \xrightarrow{2} v_2 \\ \mathscr{L}(v) & \text{if } \mathscr{L}(v) = \mathsf{End} \text{ or } \mathscr{L}(v) \in B \end{cases}$$

The function $\mathcal{T}$ constructs a type for a node $v \in V,\ v \neq o$. The approach is fairly straightforward: the node's label determines the type constructor and the subtypes are constructed recursively from the set of immediate successors of v, excluding o. The graph may contain cycles which have to be turned into recursive types. This is achieved by keeping track of the already seen nodes in the parameter η. Every node v that may be part of a cycle, i. e. $\mathscr{L}(v) \in L^S_{Con}$, binds a type variable v the first time it is encountered. The second time round the constructed time refers to the now in-scope type variable. In the recursive calls for successors of v that may not be part of a cycle with v we do not pass η on. In the last clause of we assume that every base type label corresponds to a valid base type. This assumption is reasonable for choices like `Unit` or `Int`.

Lemma 5.2. *The types produced by $\mathcal{T}$ are well-formed, that is every type is contractive and formed according to the grammar T. An argument with a label in L^S_{Con} forms a type according to the grammar S.*

Proof. By inspection of $\mathcal{T}$ every produced type is contractive. [js: incomplete] □

Lemma 5.3. *The types produced by $\mathcal{T}$ contain recursion variables only in tail recursive positions of session types.*

Proof. [js: TODO] □

5.2 Duality

We need a way to identify nodes in the LTS that represent dual types. To this end we define the notion of duality relations.

Definition 5.4. Given an LTS $(V, L, \longrightarrow)$ a relation $\mathcal{D} \subseteq V \times V$ is a *duality relation* if for any two nodes $u, v \in V$ with $u\ \mathcal{D}\ v$ it holds that

(1) $\{\mathscr{L}(u), \mathscr{L}(v)\} \in \{\{!, ?\}, \{+, \&\}\}$
(2) $u \xrightarrow{\ell} u'$ for $\ell \in L \setminus \{1\}$ implies $v \xrightarrow{\ell} v'$ such that $u'\ \mathcal{D}\ v'$
(3) $v \xrightarrow{\ell} v'$ for $\ell \in L \setminus \{1\}$ implies $u \xrightarrow{\ell} u'$ such that $u'\ \mathcal{D}\ v'$
(4) $u \xrightarrow{1} w$ if and only if $v \xrightarrow{1} w$ [js: this formulation requires a form of uniqueness for nodes]

Lemma 5.5. *Any duality relation $\mathcal{D}$ is*

symmetric $\quad u\ \mathcal{D}\ v \implies v\ \mathcal{D}\ u$
right-unique $\quad u\ \mathcal{D}\ v,\ u\ \mathcal{D}\ v' \implies v = v'$
left-unique $\quad u\ \mathcal{D}\ v,\ u'\ \mathcal{D}\ v \implies u = u'$

Proof.

(1) If requirements (1) and (4) are satisfied they will also be satisfied in the symmetric case. Condition (2) guarantees that condition (3) is satisfied for the symmetric case and vice versa.
(2) [js: TODO]
(3) Follows from symmetricity and right-uniqueness. □

Definition 5.6. Given an LTS $(V, L, \longrightarrow)$, two nodes session type nodes $u, v \in V$ are called *dual* if there exists a duality relation $\mathcal{D}$ such that $u\ \mathcal{D}\ v$.

Lemma 5.7. *Given two dual nodes u, v then $\overline{\mathcal{T}(u,\eta)} = \mathcal{T}(v,\eta)$.*

Proof. [js: TODO] □

Definition 5.8. For a node v the duals set $\mathsf{duals}(v)$ comprises the node v itself and, if it exists, the dual node of v.

5.3 Subtyping

We define a subtyping relation on the LTS representation and prove it sound with respect to the classical subtyping relation for session types [23, 24]. As with the duality of nodes before, the definition of subtyping is necessarily coinductive.

Definition 5.9 (Transition subtyping relation). Given a set of states or nodes V, a set of transition labels L, and a labelling function $\mathscr{L}$ a relation $\mathcal{R} \subseteq \mathscr{P}(V \times L \times V) \times V \times \mathscr{P}(V \times L \times V)$ is a *transition subtyping relation*[6] if for every $(\longrightarrow_A, v, \longrightarrow_B) \in \mathcal{R}$ the conditions enumerated below are met.

(1) $\mathscr{L}(v) = \&$ and $v \xrightarrow{\ell}_A v_\ell \implies v \xrightarrow{\ell}_B v_\ell$ and $(\longrightarrow_A, v_\ell, \longrightarrow_B) \in \mathcal{R}$
(2) $\mathscr{L}(v) = +$ and $v \xrightarrow{\ell}_B v_\ell \implies v \xrightarrow{\ell}_A v_\ell$ and $(\longrightarrow_A, v_\ell, \longrightarrow_B) \in \mathcal{R}$
(3) $\mathscr{L}(v) \in \{!, ?, \rightarrow\}$ and $(v \xrightarrow{\ell}_A v_\ell$ or $v \xrightarrow{\ell}_B v_\ell) \implies v \xrightarrow{\ell}_A v_\ell$ and $v \xrightarrow{\ell}_B v_\ell$
(4) $\mathscr{L}(v) \in \{!, \rightarrow\}$ and $v \xrightarrow{1}_A v_1, v \xrightarrow{2}_A v_2 \implies (\longrightarrow_B, v_1, \longrightarrow_A) \in \mathcal{R}, (\longrightarrow_A, v_2, \longrightarrow_B) \in \mathcal{R}$
(5) $\mathscr{L}(v) = ?$ and $v \xrightarrow{\ell} v_\ell \implies (\longrightarrow_A, v_\ell, \longrightarrow_B) \in \mathcal{R}$

Definition 5.10 (Transition system state subtyping). Given two transition systems T and U differing only in their transition relations $\longrightarrow_T$ and $\longrightarrow_U$: state v in T is subtype of that same state in U if there exists a transition subtyping relation $\mathcal{R}$ such that $(\longrightarrow_T, v, \longrightarrow_U) \in \mathcal{R}$. We write $v : T \preceq U$.

The main point of the subtyping relation $v : T \preceq U$ is that a &-labelled node may contain additional edges in T (1), whereas for a +-labelled node there may be fewer edges in T than in U (2). For any label in both transitions systems the type constructors for branch and choice are covariant in the target type. Requirement (3) ensures consistent transitions for nodes labelled, !, ?, →. Further labels need not be considered since the requirements for a typing LTS ensures that there are no outgoing transitions. A sending session type as well as functions are contravariant in their first argument and covariant in their second (4). A receiving session type is covariant in both of its arguments (5).

Lemma 5.11 (Soundness of subtyping). *Given two typing LTS A, B defined over the same set of nodes, transition labels, and shared labelling function, if $v : A \preceq B$ then $\mathcal{T}_A(v, \emptyset) \leq \mathcal{T}_B(v, \emptyset)$.*

Proof. [js: TODO] □

The subtyping defined above is not complete. In particular, the subtyping for end-recursive session types allows mixing partial unfolding of recursion with dropping/adding of choice and branch labels. Such subtypings are excluded by design since the result is not expressible through the subtyping of algebraic protocols described in this work.

5.4 Recursive types as protocols

This section describes how a recursive session type is translated to a valid protocol type. The types are assumed to be available in their LTS representation described previously. A node in the LTS labelled by an element of L^S_{Con} is translated to a protocol type. Other nodes are translated to types of kind $\mathbf{T}$ by embedding protocol types P as session types $!P.\mathsf{End}$.

[6] $\mathscr{P}(A)$ denotes the *power set* of some set A.

```
protocol Eps       = Eps
protocol Seq p q   = Seq p q
protocol Rep p     = Rep p (Rep p)
```

Listing 1. Auxiliary protocols used in the translation to algebraic protocols.

The translation makes use of the three singleton protocols shown in Listing 1. Protocol `Eps` is operationally a no-op, `Seq` p q sequences two protocols p and q, and `Rep` p repeats protocol p ad infinitum. They are used to encode recursive session types that do not contain branch or choice constructors. For example, the session type the session type $S_{\rightleftharpoons} = \mu\alpha.!\texttt{Int}.?\texttt{Int}.\alpha$ will be encoded as `!Rep (Seq +Int (Seq −Int Eps)).End`.

All other session types are encoded as a potentially empty sequence of sends and receives, followed optionally by a single send or receive of a global `Dispatch` protocol synthesized by the translation, and terminated by `End`. In this case, any recursion is "hidden" inside the `Dispatch` protocol. The translation of whole programs only synthesizes one such protocol. Protocol subtyping is used to restrict the set of constructors that are actually valid. For example the session type $!\texttt{Int}.\mu\alpha.+\langle \texttt{Add} : !\texttt{Int}.\alpha, \texttt{Result} : ?\texttt{Int}.\texttt{End}\rangle$ is encoded as `!Int.!Dispatch[Add, Result].End` where `Dispatch` has at least the following constructors:

```
protocol Dispatch = ...
                  | Add Int Dispatch[Add, Result]
                  | Result -Int Eps
```

The translation assumes that some total order on the nodes $\leq_V \subseteq V \times V$ is given. The order is extended lexicographically to sequences of nodes.

Definition 5.12 (Sequence rotations). For a sequence of n nodes and i with $1 \leq i \leq$ we define

$$\sigma^i(v_1 \dots v_n) = v_i v_{i+1} \dots v_n v_1 \dots v_{i-1}$$

Definition 5.13 (Minimum rotation). For a sequence s of n nodes the *minimum rotation* is some i with $1 \leq i \leq n$ such that for all j from the same range $\sigma^i(s) \leq_V \sigma^j(s)$.

The function POL maps a subset of node labels into type polarities. It is lifted to nodes such that $\text{POL}(v) = \text{POL}(\mathscr{L}(v))$ under the condition that v is suitably labelled.

$$\text{POL} : \{!, ?, +, \&\} \to \{+, -\}$$
$$\text{POL} = \begin{cases} !, + \mapsto + \\ ?, \& \mapsto - \end{cases}$$

Nodes in the LTS are translated to algebraic types through TY and its mutually recursive helpers defined in Figure 9. As with the translation of graph nodes to end-recursive session types we assume that any base type label corresponds to a valid algebraic base type such as, for example, `Int`. Nodes labelled End are translated to `End`. $\to$-labelled nodes are turned into function types by translating domain and codomain recursively. Because the LTS may not contain any cycles involving non-session type nodes these recursive calls are guaranteed to terminate (assuming the last case of TY, discussed next, is are guaranteed to terminate as well).

The translation of session types is more involved. The main complication arises from the need to translate recursive session types not involving choice or branch constructors into some canonical form. We call recursive session types of the form $\mu\alpha.\sharp_1 T_1 \dots \sharp_n T_n.\alpha$ (where $\forall i.\, \sharp_i \in \{!, ?\}$) *linearly recursive.* The recursion of such types is encoded through the `Rep` protocol. It receives as an argument the protocol to repeat which we build up nesting multiple instances of `Seq`. As an example,

$$\textsc{ty}(v) = \begin{cases} \mathscr{L}(v) & \text{if } \mathscr{L}(v) \in L^B_{Con} \text{ or } \mathscr{L}(v) = \mathsf{End} \\ \textsc{ty}(v_1) \to \textsc{ty}(v_2) & \text{if } \mathscr{L}(v) = \to, v \xrightarrow{1} v_1, v \xrightarrow{2} v_2 \\ !{\pm}\textsc{seq}^{\pm}(\textsc{collect}(\varepsilon, v)).\mathsf{End} & \text{if } \mathscr{L}(v) \in L^S_{Con} \setminus \{\mathsf{End}\}, \pm = \textsc{pol}(v) \end{cases}$$

$$\textsc{collect}(x, v) = \begin{cases} (xv_1 \dots v_{j-1}, v_j \dots v_n v_1 \dots v_{j-1}) & \text{if } v_1 \xrightarrow{2} \cdots v_n \xrightarrow{2} v_1 \text{ where } \forall i.\, v_i = v \Leftrightarrow i = 1 \\ & \text{where } j \text{ is the minimal rotation of } v_1 \dots v_n \\ (x, \varepsilon) & \text{else if } \mathscr{L}(v) = \mathsf{End} \\ (xv, \varepsilon) & \text{else if } \mathscr{L}(v) \in \{+, \&\} \\ \textsc{collect}(xv, v_2) & \text{else if } \mathscr{L}(v) \in \{!, ?\}, v \xrightarrow{2} v_2 \end{cases}$$

$$\textsc{seq}^{\pm}(x, y) = \begin{cases} \mathsf{Eps} & \text{if } x = y = \varepsilon \\ \mathsf{Rep}\ \textsc{seq}^{\pm}(y, \varepsilon) & \text{if } x = \varepsilon, y \neq \varepsilon \\ \mathsf{Seq}\ {\pm}\textsc{pol}(v)\textsc{ty}(v_1)\ \textsc{seq}^{\pm}(x', y) & \text{if } x = vx', \mathscr{L}(v) \in \{!, ?\}, v \xrightarrow{1} v_1 \\ {\pm}\textsc{pol}(v)\textsc{p}(v)[I] & \text{if } x = vx', \mathscr{L}(v) \in \{+, \&\}, I = \{\ell \in L \mid \exists v'.\, v \xrightarrow{\ell} v'\} \end{cases}$$

$$\textsc{p}(v) = \begin{cases} P_{v'} & \text{if } \mathscr{L}(v) = \& \text{ and there exists a dual node } v' \\ P_v & \text{else} \end{cases}$$

Fig. 9. Translation functions from LTS nodes to algebraic types

consider the type $\mu\alpha.!\mathtt{Int}.?\mathtt{Int}.\alpha$ that is (slightly simplified) encoded as `!Rep (Seq Int −Int)`. After one `send` (and any necessary `select` operations) we are left with `?Int.`$\mu\alpha.$`!Int.?Int.`α and `?Int.!Rep (Seq Int −Int)` respectively. The former is equivalent to $\mu\alpha.?\mathtt{Int}.!\mathtt{Int}.\alpha$ but the latter is not equivalent to `!Rep (Seq −Int Int)`. It is important that these equivalences are preserved by the translation.

Given a node v and a (potentially empty) sequence of nodes x COLLECT returns a pair of two node sequences. The first component contains the sequence of nodes corresponding to send and receives, and potentially terminated by a node labelled either & or +. If not empty the second component will contain a sequence of nodes comprising a linearly recursive cycle. x is prepended as a prefix to the first component. The first clause of COLLECT contains the solution to the problem outlined above. Using the total order on the nodes V the minimum rotation j is determined. In the first component part of the cycle, up to but not including node the j-th node, is returned. The second component of the result consists of the cycle but rotated to start with the j-th node.

If the node v given to COLLECT is not part of a linearly recursive cycle there are three further cases to consider. If the node's label is End, &, or + the collection is terminated. v is included in the result if it is not an End node. In the final case v is appended to x in a recursive call to COLLECT for the node reachable via a 2 transition from v.

SEQ$^{\pm}$ assembles the sequences returned from COLLECT into a protocol type. If both sequences are empty a base case will have been reached and the protocol `Eps` is returned. If only the first sequence is empty the protocol to be built must correspond to a linearly recursive type. As such, the second argument is moved into the first and the result is used as an argument to the `Rep` protocol. If the first sequence is not empty and starts with a node labelled ! or ? the payload type is turned into an algebraic type and the remainder of the sequence is processed recursively. If the head of the sequence is labelled + or & instead [js: we use multiple protocols]

The polarity parameter $\pm$ to SEQ combined with the polarities returned by POL(v) produce dual session types for dual nodes.

The recursion between TY and SEQ is guaranteed to terminate because every call from SEQ back into TY gives as input a node that was reached via a 1 transition. Such transitions cannot be part of any cycles.

For every node v labelled + or & and a potentially existing dual node v' we define a protocol

$$\texttt{protocol}\ \textsc{p}(v) = \{C_\ell\}$$

The protocol `Dispatch` is defined as

$$\texttt{protocol Dispatch} = \left\{C_\ell\ \textsc{proto}^{\textsc{pol}(v)}(v') \;\middle|\; v, v' \in V,\ \ell \in L \setminus \{1,2\},\ v \xrightarrow{\ell} v'\right\}$$

Lemma 5.14 (Well-formedness of generated types).

(1) *For all $v \in V$ it is derivable that $\Delta \vdash \textsc{ty}(v) \Leftarrow \mathbf{T}$. In particular, $\mathscr{L}(v) \in L^S_{Con}$ implies $\Delta \vdash \textsc{ty}(v) \Leftarrow \mathbf{S}$ is derivable.*

(2) *For all $v \in V$ where $\mathscr{L}(v) \in L^S_{Con}$ and all $\theta : V \to \{0,1,2\}$ it is derivable that $\Delta \vdash \textsc{proto}^{\pm}(v,\theta) \Leftarrow \mathbf{P}$ and $\Delta \vdash \textsc{step}^{\pm}(v,\theta) \Leftarrow \mathbf{P}$.*

In either case it is assumed that Δ contains at least the protocol definitions for `Eps`, `Seq`, `Rep`, `Dispatch`.

Lemma 5.15. *Given two LTS $A = (V, L, \longrightarrow_A)$ and $B = (V, L, \longrightarrow_B)$ defined over the same set of nodes, transition labels, and the same labelling function, for all $v \in V$ it holds that $\textsc{Sub}_v(\longrightarrow_A, \longrightarrow_B)$ if and only if $\textsc{ty}_A(v) \leq \textsc{ty}_B(v)$.*

Lemma 5.16. *For all pairs of session type nodes u, v it holds that u and v are dual nodes if and only if $\mathrm{nrm}^-(\textsc{ty}(u)) = \mathrm{nrm}^+(\textsc{ty}(v))$.*

5.5 Expressions and processes

After having specified how the type language of end-recursive session types can be embedded in algebraic protocols we now tackle the translation of expressions and processes. The translation mainly needs to insert additional `select` and `match` statements to handle the auxiliary protocols introduced by the type translation.

Due to the way session types are translated a `select Seq` or corresponding `match` must be inserted in front of each `send` and `receive`. Already existing `select` and `match` operations correspond directly to their counterparts on the `Dispatch` protocol. Since `End` is translated to itself but `!Int.End` is translated to `!Seq Int Eps.End` which leaves `!Eps.End` after the communication step `select` and `match` statements must be introduced to deal with `Eps` protocols. Similar handling is necessary for non-branching recursive session types like $\mu\alpha.!\texttt{Int}.\alpha$.

6 Implementation

The implementation of AlgST consists of a type checker and an interpreter, both written in Haskell. Aiming at supporting more realistic programs, the implementation features pragmatic extensions, including extensions to the formal system and a simple module system for name space management.

Type checking. The syntax of the implementation closely matches the examples in this paper. Some places need additional kind annotations because the type system of the implementation is not restricted to linear types. Its kind structure distinguishes between linear and unrestricted variants of the kinds **S** and **T**. Subkinding allows us to subsume unrestricted versions of types to linear ones, that is, $\mathbf{T}^{\mathbf{UN}} < \mathbf{T}^{\mathbf{LIN}}$. This subkinding generates a simple subtyping relation that essentially allows us to provide an unrestricted value where a linear one is expected. The protocol kind **P** does not split in two versions, but still any type can be lifted to a protocol type. Almeida et al. [4] investigate the metatheory of a system with a similar kind structure (without the kind **P**).

The type checker implementation closely follows the bidirectional rules given in the paper. It necessarily contains an implementation of the type normalization algorithm shown in Figure 3.

```
--- protocol and type in AlgST syntax ---
protocol Repeat x = More x (Repeat x) | Quit
?Repeat Int . !(Char, End) . End
--- corresponding type in FreeST syntax ---
(rec repeat0 : 1S . &{More : ?Int ; repeat0 ; Skip,
                      Quit : Skip}) ; (!(Char, End) ; End)
--- example of an equivalent AlgST type ---
Dual (!Repeat Int . ?(Char, End) . Dual End)
--- example of a non-equivalent AlgST type ---
?Repeat String . !(Char, End) . End
```

Fig. 10. An AlgST type instance, its FreeST counterpart and examples of equivalent and non-equivalent AlgST types, following the rules of our test suite generator.

This algorithm is extended to cater for the type isomorphism $\forall\alpha.(T \to U) \cong T \to \forall\alpha.U$ (if α not free in T). In this way, type applications can be placed more liberally as long as their sequence is preserved.

The normalization algorithm is invoked exactly as indicated by the algorithmic typing rules in Figure 5. By design, the rules keep the typing assumptions and the outcome of inference in normal form to minimize the number of times normalization is invoked.

The type checker includes further checking rules to enable writing some lambda abstractions without type annotations. These rules are adaptions of well-known rules [18]. For example:

$$\text{E-Abs'}\quad \frac{\Delta \mid \Gamma_1, x\colon T \vdash e \Leftarrow U \mid \Gamma_2 \qquad x \notin \Gamma_2}{\Delta \mid \Gamma_1 \vdash \lambda x.e \Leftarrow T \to U \mid \Gamma_2} \qquad \text{E-App'}\quad \frac{\Delta \mid \Gamma_1 \vdash e_2 \Rightarrow T \mid \Gamma_2 \qquad \Delta \mid \Gamma_2 \vdash e_1 \Leftarrow T \to U \mid \Gamma_3}{\Delta \mid \Gamma_1 \vdash e_1\ e_2 \Leftarrow U \mid \Gamma_3}$$

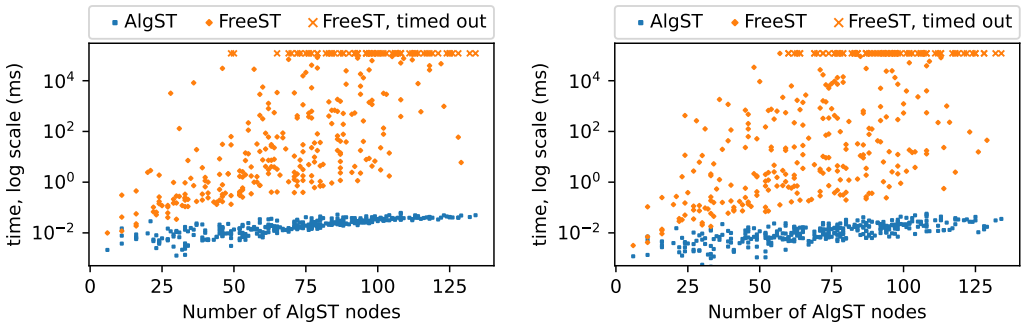
The implementation fully supports the overloaded use of data constructors as protocol operators. We can perform pattern matching against the constructors of a `List` datatype and use the same constructors in selecting alternatives of the `!List` protocol. Moreover, the `case` operator is also overloaded to serve as the `match` operator.

Interpretation. The interpreter employs standard techniques for the functional part and for representing values at run time. It uses one universal type with constructors for each type in the language. Processes in AlgST are mapped to Haskell threads. A fork in the language is implemented using `forkIO` in Haskell. The interpreter is invoked recursively for the new thread.

Communication channels are implemented using shared-memory abstractions from Haskell's `Control.Concurrent` library. Synchronous communication (as in this paper) is implemented by pairs of `MVars`, a semaphore-like concurrent datastructure [30] that behaves like a buffer of size one. The implementation has an option to switch to asynchronous channels using bounded queues (`TBQueue`) from the `stm` library.

As channels are implemented in shared memory, the `send` primitive is naturally polymorphic. Internally, the interpreter mediates values of the universal type mentioned above between `send` and `receive` operations.

Benchmarking. We substantiate our claim that replacing a worst-case superlinear algorithm for type equivalence by a linear one yields actual run-time improvements with an experiment. We compare AlgST's type equivalence algorithm with FreeST, a freely available [21] programming language containing an implementation of type equivalence for polymorphic context-free session



(a) equivalent test cases

(b) non-equivalent test cases

Fig. 11. Execution time of the AlgST and FreeST type equivalence algorithms on the equivalent and non-equivalent test cases, represented in log scale as a function of the number of AlgST nodes in the AST.

types [3, 4]. As the type checker regularly requires type equivalence checks, the performance of the equivalence algorithm should be understood as a lower bound of the type checker performance.

To create a collection of equivalent and non-equivalent test cases, we implemented a generator of instances of AlgST types. An instance comprises a set of mutually recursive algebraic protocols and a session type referring to them. We carefully restrict protocols and types so that a translation from AlgST instances to FreeST types is possible: the generator avoids polymorphic and nested recursion and restricts the occurrences of the negation operator to the top level of protocol constructor arguments. For each instance T, we randomly apply the properties of normalization to generate an equivalent AlgST type T' (see Figure 3). The pairs (T, T') constitute our test suite of equivalent types. Non-equivalent tests are obtained from each T by either introducing an additional quantifier, or changing a sub-part of the type to something of the same kind. Possible replacements are the quantified type variables and the types built into AlgST, such as `Int` or `String`. The AlgST type is translated to a session type in FreeST. Protocols are translated inline at every point of use as recursive branch or choice types, depending on wether it appears in a sending or receiving context. For single constructor types, the translation omits the constructor tag. The arguments of the constructors are translated into nested sequences of single interactions. See Figure 10 for an example.

Benchmarks were run using the gauge package v0.2.5 [40] with a timeout of 2 minutes. AlgST was compiled using GHC 9.2.7, FreeST used GHC 8.10.7. Each test suite comprises 324 tests. FreeST incurred 69 timeouts for the positive tests and 77 timeouts for the negative tests. Figure 11 compares the execution time of the AlgST and FreeST type equivalence algorithms for the equivalent and non-equivalent test cases. The plots substantiate the differences in the execution time of FreeST and AlgST, distinguishing the linear evolution of AlgST and the exponential behavior of FreeST as a function of the number of nodes in the abstract syntax tree of the AlgST type.

7 Related Work

This survey of related work concentrates on binary session types and their treatment of recursion. Many works in the area concentrate on different aspects and treat recursion informally as an add-on. In particular, the early work of Honda [28], Takeuchi et al. [51] did not consider recursive types. The school of session types that takes its foundations from the sequent calculus for dual intuitionistic logic [13, 14] initially focuses on finite behaviors. While recursion would break

the logical consistency of their system, they consider dependently typed variants that include replication [45].

Session type systems with recursion. Generally, recursive types come in two flavors, equirecursive and isorecursive [1], and the impact of these different approaches on type checking is well-studied [46]. They have been found to be equally expressive [43]. Most session type systems in the literature support equirecursive session types restricted to tail recursion and treat them informally.

Recursive types have first been introduced to session types by Honda et al. [29] in the context of π-calculus. Their system relies on equirecursion and is syntactically restricted to tail recursion. Gay and Hole [22, 23] study subtyping in the same setting using a coinductive definition. Castagna et al. [15] consider recursive session types as infinite regular trees. Their types are tagless in the sense that choices are made according to the set of accepted values and the viability of the continuation. The functional session type calculus of Gay and Vasconcelos [24] includes subtyping and equirecursive types, but the latter are restricted to tail-recursive session types. Fundamentals of session types [56] proposes a range of π-calculus-based systems with increasing complexity. One particular focus is the treatment of replication and unrestricted channel ends, which requires recursive types. In this work, recursive types are modeled by infinite regular trees, represented using the familiar μ notation with contractive, tail recursive bodies.

Toninho et al. [54] extend the linear-logic based line of work and focus on the combination of functions and processes including recursive processes and consequently recursive types for processes and functions. The examples indicate the equirecursive variant. Apparently, there is no restriction in the use of recursion and sequencing is implemented using the tensor $\otimes$. Toninho et al. [55] cover a different aspect of equirecursion. They consider corecursive protocol definitions and analyze their productivity. The main question they pursue is when a corecursive protocol produces the next value after finitely many internal steps. SILL [44] is a process-based language design that integrates linear and affine types as well as synchronous and asynchronous communication. It builds directly on the linear logic interpretation of session types. Its examples make use of polymorphism and equirecursive types in the form of tail recursive type equations, neither of which are reflected in the type theoretical presentation. Bartoletti et al. [9] offer a semantic analysis of session types on the basis of labeled transition systems. Their system features equirecursion in tail position.

Lindley and Morris [33] define two functional session type systems with recursion, μGV and μCP, and show that they are related by typing- and semantics-preserving translations. So we only compare to μGV in the following. μGV is based on a simply-typed linear lambda calculus extended with notions of recursion and corecursion. These are defined as least and greatest (respectively) fixed points of strictly positive functors — analogous to a logically meaningful version of algebraic (co-) datatypes — with generic fold and unfold operators. Finally, they lift this isorecursive treatment of recursive data to the level of session types by encoding branching and recursion, mapping the corresponding data functors over session types and wrapping them in promises. We conclude that μGV features session types with isorecursion and that it could be used to encode the simply-typed fragment of AlgST, but without parameterized data types and without the over direction. Where μGV uses promises (i.e., a separate channel) to encode the argument to the fold of the recursive type (i.e., the constructor arguments of an algebraic datatype), AlgST transmits the body of the fold on the same channel, thus avoiding the overhead of channel creation. Moreover, μGV is mainly a theoretical contribution whereas a full implementation of AlgST is available.

Context-free and nested session type systems. Thiemann and Vasconcelos [52] proposed context-free session types to enable the type-safe encoding of non-regular protocols like the serialization of tree structures or the stack protocol from the introduction without encurring the overhead of

session initiation. They changed the asymmetric structure of the traditional session type syntax $?T.S$ and $!T.S$, where an item of type T is communicated and the session continues with S, to a symmetric structure where session types can be built from the atomic transmissions, $?T$ and $!T$, choice and branching, $S_1 \oplus S_2$ and $S_1 \,\&\, S_2$, and a monoidal sequential composition operator $S_1; S_2$ with unit `Skip`. This design, together with equirecursive sessions, leads to a number of technical problems. First, types come with a non-trivial equational theory: sequential composition forms a monoid with unit `Skip` and it distributes over choice and branching. Second, polymorphic recursion is needed as a recursive call may have to be instantiated with different continuation sessions. Third, the interaction between equirecursion and the omission of the restriction to tail recursion gives rise to a difficult, but decidable, type equivalence relation. Subsequent work investigated tractable algorithms for type equivalence [2, 3]. Almeida et al. [4] integrate context-free session types with explicit polymorphism in the style of System F. Recent work extends this approach with higher-order types [16].

Padovani [41, 42] proposes a different approach to context-free session types embodied in a functional session calculus called FuSe$^{\{\}}$. This calculus uses the type language of context-free session types, but sidesteps the problems with the sequencing operator and type equivalence by introducing a resumption operator $\{e\}_c$ that deconstructs a sequence of sessions on channel $c : S_1; S_2$. It does so by retyping the channel to $c : S_1$ for evaluating e, which must return the same channel after finishing S_1. More formally, the resumption obeys some kind of frame rule: if $c : S_1 \vdash e : T \times 1$, then $c : S_1; S_2 \vdash \{e\}_c : T \times S_2$. The resumption temporarily removes the continuation S_2 from the session type and sticks it back on afterwards. The type system has means to make sure that the channel returned by e is identical to c, which is required for soundness. The type 1 does not signify the end of the session, but rather that some known part of the session has been processed and the session will be continued in the context (of the resumption). The management of the choice and branch operations is very close to AlgST. FuSe$^{\{\}}$ implements choice by transmitting the constructor tag in a variant type that selects the different continuations. Its branch operator matches on the received constructor and thus selects the desired continuation. However, the handling of protocol constructors in AlgST includes the construction of the continuation type from the types and directions given in the protocol declaration. FuSe$^{\{\}}$ has no equivalent to a protocol declaration nor to the type operator - to swap the direction of a transmission. Padovani [42] implements FuSe$^{\{\}}$ in OCaml and demonstrates that context-free session types are amenable to type inference, if the programmer helps with judicious placement of resumptions. Support for equirecursion is already built into OCaml's type inference engine. It is also shown that subtyping is undecidable.

Das et al. [17] propose the metatheory of nested session types featuring parametrized definitions of session types with nested polymorphic recursion. At first glance, the nested session type $\texttt{List}[\alpha] = \oplus\{\texttt{nil}: \texttt{end}, \texttt{cons}: \alpha \otimes \texttt{List}[\alpha]\}$ is very similar to the algebraic protocol `protocol List` α = `Nil | Cons` α (`List` α), but there are significant differences. On the algebraic side, we can materialize the `List` protocol in a type that sends or receives a list of integers, while the nested type `List` is already committed to a concrete direction. Although type nesting enables to capture the modularity characterizing algebraic sessions, there are several features that distinguish AlgST from nested session types: Das et al. propose a structural and equirecursive, rather than a nominal, view of type definitions, and interpret types coinductively, rather than inductively. These features have significant impact on the complexity of the equivalence problem for nested session types and on its implementation: Das et al. present a sound but incomplete algorithm for type equivalence. By considering a nominal interpretation of algebraic session types, we reduced the complexity of the equivalence problem from doubly-exponential [17] to linear and designed a sound and complete algorithm to verify type equivalence. The examples in section 2 of Das et al. [17] translate into AlgST and type check with our implementation.

Gay et al. [26] consider equirecursive protocols defined by different kinds of systems of equations (with and without parameters). Imposing different restrictions on the parameterization yields different classes of protocols with decision problems for type equivalence ranging from polytime to undecidable. Their largest decidable class of pushdown session types is shown to be equivalent to nested session types, which we can model with AlgST.

Compared to all competiting systems [4, 17, 42, 53], AlgST's materialization and directional operators are unique. These operators enable the construction of protocols without committing to a particular direction.

Linear and unrestricted types. Our calculus is mostly linear, except that we allow unrestricted bindings for recursive functions, which would be rendered useless otherwise. Our implementation goes well beyond that in allowing fully fledged unrestricted computations besides ensuring that session typed channels are treated linearly. There is some work on recursion and linearity in the context of lambda calculus [5] adding iterators on linear natural numbers to the calculus, thereby avoiding the need for a rule like E-Var$\star$.

System F$^\circ$ [35] extends System F with kinds to integrate polymorphism with linear and unrestricted types. This work serves as a blueprint for much subsequent work in this area including LFST and FreeST (see below), as well as our implemented language. They categorize all objects, including functions, into one-use or many-use resources. This style can be traced back to Walker [57]. LFST [34] formalizes the session type system of Links, which features a mix of linear and unrestricted types, embedding a standard functional programming language. The substructural behavior of types is specified by a sophisticated kind structure. The kinding used in our implementation is inspired by LFST, though LFST has additional kinds to describe row types that we do not support. Almeida et al. [4] consider an integration of context-free session types with System F. Their type system includes a kinding discipline that manages linear and unrestricted versions of ordinary types and message types (possible payloads for the send and receive operations). Our implemented system extends the kind structure in a similar way.

Linear Haskell (LH) [12] is a proposal to integrate linear types with stock functional programming. While the work discussed so far characterizes resources, LH supports the lollipop type $S \multimap T$ which classifies functions that use their argument exactly once. Originally, LH forced the programmer to allocate resources using continuations. This restriction has been fixed in subsequent work [50]. There is further work on integrating linear and dependent types (e.g., [32]) which relies on bifurcating the language in two calculi connected by an adjunction.

8 Conclusion

Algebraic protocols combine naming and recursion with choice and sequence type constructors known from session types. Like algebraic datatypes, they offer parameterization and mutual recursion. Unlike previous work on session types, algebraic protocols do not commit to a particular direction of communication, even if direction reversal can be specified at the type level. Algebraic protocols and session types subsume the expressive power of context-free and nested session type systems while reducing the complexity of type checking from doubly-exponential to linear time.

Software Availability

The proposed system and implementation are publicly available as an artifact [49]. The interested reader can follow the steps in the artifact documentation to set up a pre-built docker image or a self-built docker image. Besides the implementation of the type checker and the interpreter, we provide programs for all the examples in the paper and examples that establish a comparison with

related work [17, 52]. In the artifact, the reader can also find instructions on how to reproduce our benchmarking results for type equivalence.

References

[1] Martín Abadi and Marcelo P. Fiore. Syntactic considerations on recursive types. In *11th Annual IEEE Symposium on Logic in Computer Science*, pages 242–252, New Brunswick, New Jersey, USA, 1996. IEEE Computer Society. doi: 10.1109/LICS.1996.561324.

[2] Bernardo Almeida, Andreia Mordido, and Vasco T. Vasconcelos. Freest: Context-free session types in a functional language. In Francisco Martins and Dominic Orchard, editors, *PLACES@ETAPS 2019*, volume 291 of *EPTCS*, pages 12–23, Prague, Czech Republic, 2019. doi: 10.4204/EPTCS.291.2.

[3] Bernardo Almeida, Andreia Mordido, and Vasco T. Vasconcelos. Deciding the bisimilarity of context-free session types. In Armin Biere and David Parker, editors, *TACAS 2020*, volume 12079 of *Lecture Notes in Computer Science*, pages 39–56, Dublin, Ireland, 2020. Springer. doi: 10.1007/978-3-030-45237-7_3.

[4] Bernardo Almeida, Andreia Mordido, Peter Thiemann, and Vasco T. Vasconcelos. Polymorphic lambda calculus with context-free session types. *Information and Computation*, 2022. URL https://doi.org/10.1016/j.ic.2022.104948.

[5] Sandra Alves, Maribel Fernández, Mário Florido, and Ian Mackie. Linearity and iterator types for gödel's system. *High. Order Symb. Comput.*, 23(1):1–27, 2010. doi: 10.1007/s10990-010-9060-x.

[6] Robert Atkey. Syntax and semantics of quantitative type theory. In Anuj Dawar and Erich Grädel, editors, *LICS 2018*, pages 56–65, Oxford, UK, 2018. ACM. doi: 10.1145/3209108.3209189. URL https://doi.org/10.1145/3209108.3209189.

[7] Stephanie Balzer and Frank Pfenning. Manifest sharing with session types. *Proc. ACM Program. Lang.*, 1(ICFP): 37:1–37:29, 2017. doi: 10.1145/3110281.

[8] Gilles Barthe and Maria João Frade. Constructor subtyping. In S. Doaitse Swierstra, editor, *Programming Languages and Systems, 8th European Symposium on Programming, ESOP'99, Held as Part of the European Joint Conferences on the Theory and Practice of Software, ETAPS'99, Amsterdam, The Netherlands, 22-28 March, 1999, Proceedings*, volume 1576 of *Lecture Notes in Computer Science*, pages 109–127. Springer, 1999. doi: 10.1007/3-540-49099-X_8.

[9] Massimo Bartoletti, Alceste Scalas, and Roberto Zunino. A semantic deconstruction of session types. In Paolo Baldan and Daniele Gorla, editors, *CONCUR 2014*, volume 8704 of *Lecture Notes in Computer Science*, pages 402–418, Rome, Italy, 2014. Springer. doi: 10.1007/978-3-662-44584-6_28.

[10] Giovanni Bernardi and Matthew Hennessy. Using higher-order contracts to model session types (extended abstract). In *CONCUR*, volume 8704 of *Lecture Notes in Computer Science*, pages 387–401, Rome, Italy, 2014. Springer. doi: 10.1007/978-3-662-44584-6_27.

[11] Giovanni Bernardi and Matthew Hennessy. Using higher-order contracts to model session types. *Logical Methods in Computer Science*, 12(2), 2016. doi: 10.2168/LMCS-12(2:10)2016.

[12] Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. Linear haskell: practical linearity in a higher-order polymorphic language. *Proc. ACM Program. Lang.*, 2(POPL):5:1–5:29, 2018. doi: 10.1145/3158093. URL https://doi.org/10.1145/3158093.

[13] Luís Caires and Frank Pfenning. Session types as intuitionistic linear propositions. In Paul Gastin and François Laroussinie, editors, *CONCUR 2010*, volume 6269 of *Lecture Notes in Computer Science*, pages 222–236, Paris, France, 2010. Springer. doi: 10.1007/978-3-642-15375-4_16.

[14] Luís Caires, Frank Pfenning, and Bernardo Toninho. Linear logic propositions as session types. *Math. Struct. Comput. Sci.*, 26(3):367–423, 2016. doi: 10.1017/S0960129514000218.

[15] Giuseppe Castagna, Mariangiola Dezani-Ciancaglini, Elena Giachino, and Luca Padovani. Foundations of session types. In António Porto and Francisco Javier López-Fraguas, editors, *PPDP 2009*, pages 219–230, Coimbra, Portugal, 2009. ACM. doi: 10.1145/1599410.1599437.

[16] Diana Costa, Andreia Mordido, Diogo Poças, and Vasco T. Vasconcelos. Higher-order context-free session types in system F. In Marco Carbone and Rumyana Neykova, editors, *Proceedings of the 13th International Workshop on Programming Language Approaches to Concurrency and Communication-cEntric Software, PLACES@ETAPS 2022, Munich, Germany, 3rd April 2022*, volume 356 of *EPTCS*, pages 24–35, 2022. doi: 10.4204/EPTCS.356.3. URL https://doi.org/10.4204/EPTCS.356.3.

[17] Ankush Das, Henry DeYoung, Andreia Mordido, and Frank Pfenning. Nested session types. In Nobuko Yoshida, editor, *Programming Languages and Systems - 30th European Symposium on Programming, ESOP 2021*, volume 12648 of *Lecture Notes in Computer Science*, pages 178–206, Luxembourg City, Luxembourg, 2021. Springer. doi: 10.1007/978-3-030-72019-3_7.

[18] Jana Dunfield and Neel Krishnaswami. Bidirectional typing. *ACM Comput. Surv.*, 54(5):98:1–98:38, 2021. doi: 10.1145/3450952.

[19] Simon Fowler, Wen Kokke, Ornela Dardha, Sam Lindley, and J. Garrett Morris. Separating sessions smoothly. In Serge Haddad and Daniele Varacca, editors, *32nd International Conference on Concurrency Theory, CONCUR 2021, August 24-27, 2021*, volume 203 of *LIPIcs*, pages 36:1–36:18, Virtual Conference, 2021. Schloss Dagstuhl - Leibniz-Zentrum für Informatik. doi: 10.4230/LIPIcs.CONCUR.2021.36.

[20] Simon Fowler, Wen Kokke, Ornela Dardha, Sam Lindley, and J. Garrett Morris. Separating sessions smoothly. *CoRR*, abs/2105.08996, 2021. URL https://doi.org/10.48550/arXiv.2105.08996.

[21] FreeST. The freest programming language. http://rss.di.fc.ul.pt/tools/freest/, (last accessed March 2023).

[22] Simon J. Gay and Malcolm Hole. Types and subtypes for client-server interactions. In S. Doaitse Swierstra, editor, *Programming Languages and Systems, 8th European Symposium on Programming, ESOP'99*, volume 1576 of *Lecture Notes in Computer Science*, pages 74–90, Amsterdam, The Netherlands, 1999. Springer. doi: 10.1007/3-540-49099-X_6.

[23] Simon J. Gay and Malcolm Hole. Subtyping for session types in the pi calculus. *Acta Informatica*, 42(2-3):191–225, 2005. doi: 10.1007/s00236-005-0177-z.

[24] Simon J. Gay and Vasco Thudichum Vasconcelos. Linear type theory for asynchronous session types. *J. Funct. Program.*, 20(1):19–50, 2010. doi: 10.1017/S0956796809990268.

[25] Simon J. Gay, Peter Thiemann, and Vasco T. Vasconcelos. Duality of session types: The final cut. In Stephanie Balzer and Luca Padovani, editors, *Proceedings of the 12th International Workshop on Programming Language Approaches to Concurrency- and Communication-cEntric Software, PLACES@ETAPS 2020, Dublin, Ireland, 26th April 2020*, volume 314 of *EPTCS*, pages 23–33, 2020. doi: 10.4204/EPTCS.314.3.

[26] Simon J. Gay, Diogo Poças, and Vasco T. Vasconcelos. The different shades of infinite session types. In Patricia Bouyer and Lutz Schröder, editors, *FOSSACS 2022*, volume 13242 of *Lecture Notes in Computer Science*, pages 347–367, Munich, Germany, 2022. Springer. doi: 10.1007/978-3-030-99253-8_18.

[27] Cordelia V. Hall, Kevin Hammond, Simon L. Peyton Jones, and Philip Wadler. Type classes in haskell. *ACM Trans. Program. Lang. Syst.*, 18(2):109–138, 1996. doi: 10.1145/227699.227700.

[28] Kohei Honda. Types for dyadic interaction. In Eike Best, editor, *CONCUR '93*, volume 715 of *Lecture Notes in Computer Science*, pages 509–523, Hildesheim, Germany, 1993. Springer. doi: 10.1007/3-540-57208-2_35.

[29] Kohei Honda, Vasco Thudichum Vasconcelos, and Makoto Kubo. Language primitives and type discipline for structured communication-based programming. In Chris Hankin, editor, *Programming Languages and Systems - ESOP'98, 7th European Symposium on Programming*, volume 1381 of *Lecture Notes in Computer Science*, pages 122–138, Lisbon, Portugal, 1998. Springer. doi: 10.1007/BFb0053567.

[30] Simon L. Peyton Jones, Andrew D. Gordon, and Sigbjørn Finne. Concurrent Haskell. In Hans-Juergen Boehm and Guy L. Steele Jr., editors, *POPL'96*, pages 295–308, St. Petersburg Beach, Florida, USA, 1996. ACM Press. doi: 10.1145/237721.237794.

[31] Steve Klabnik, Carol Nichols, and Rust Community. The Rust programming language. https://doc.rust-lang.org/book/, 2018.

[32] Neelakantan R. Krishnaswami, Pierre Pradic, and Nick Benton. Integrating linear and dependent types. In Sriram K. Rajamani and David Walker, editors, *POPL 2015*, pages 17–30, Mumbai, India, 2015. ACM. doi: 10.1145/2676726.2676969.

[33] Sam Lindley and J. Garrett Morris. Talking bananas: structural recursion for session types. In Jacques Garrigue, Gabriele Keller, and Eijiro Sumii, editors, *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, ICFP 2016*, pages 434–447, Nara, Japan, 2016. ACM. doi: 10.1145/2951913.2951921.

[34] Sam Lindley and J. Garrett Morris. Lightweight functional session types. In Simon Gay and António Ravara, editors, *Behavioural Types: from Theory to Tools*. River Publishers, Gistrup, Denmark, 2017. Extended version at https://homepages.inf.ed.ac.uk/slindley/papers/fst-extended.pdf.

[35] Karl Mazurak, Jianzhou Zhao, and Steve Zdancewic. Lightweight linear types in system fdegree. In Andrew Kennedy and Nick Benton, editors, *TLDI 2010*, pages 77–88, Madrid, Spain, 2010. ACM. doi: 10.1145/1708016.1708027. URL https://doi.org/10.1145/1708016.1708027.

[36] Conor McBride. I got plenty o' nuttin'. In Sam Lindley, Conor McBride, Philip W. Trinder, and Donald Sannella, editors, *A List of Successes That Can Change the World - Essays Dedicated to Philip Wadler on the Occasion of His 60th Birthday*, volume 9600 of *Lecture Notes in Computer Science*, pages 207–233, Edinburgh, Scotland, 2016. Springer. doi: 10.1007/978-3-319-30936-1_12. URL https://doi.org/10.1007/978-3-319-30936-1_12.

[37] Fabrizio Montesi and Marco Peressotti. Linear logic, the π-calculus, and their metatheory: A recipe for proofs as processes. *CoRR*, abs/2106.11818, 2021. URL https://arxiv.org/abs/2106.11818.

[38] Andreia Mordido, Janek Spaderna, Peter Thiemann, and Vasco T. Vasconcelos. Parameterized algebraic protocols. *Proc. ACM Program. Lang.*, 7(PLDI):1389–1413, 2023. doi: 10.1145/3591277. URL https://doi.org/10.1145/3591277.

[39] Martin Odersky, Olivier Blanvillain, Fengyun Liu, Aggelos Biboudis, Heather Miller, and Sandro Stucki. Simplicitly: foundations and applications of implicit function types. *Proc. ACM Program. Lang.*, 2(POPL):42:1–42:29, 2018. doi: 10.1145/3158130.

[40] Bryan O'Sullivan. gauge: small framework for performance measurement and analysis. https://hackage.haskell.org/package/gauge-0.2.5/, (last accessed March 2023).

[41] Luca Padovani. Context-free session type inference. In Hongseok Yang, editor, *Programming Languages and Systems - 26th European Symposium on Programming, ESOP 2017*, volume 10201 of *Lecture Notes in Computer Science*, pages 804–830, Uppsala, Sweden, 2017. Springer. doi: 10.1007/978-3-662-54434-1_30.

[42] Luca Padovani. Context-free session type inference. *ACM Trans. Program. Lang. Syst.*, 41(2):9:1–9:37, 2019. doi: 10.1145/3229062.

[43] Marco Patrignani, Eric Mark Martin, and Dominique Devriese. On the semantic expressiveness of recursive types. *Proc. ACM Program. Lang.*, 5(POPL):1–29, 2021. doi: 10.1145/3434302.

[44] Frank Pfenning and Dennis Griffith. Polarized substructural session types. In Andrew M. Pitts, editor, *FoSSaCS 2015*, volume 9034 of *Lecture Notes in Computer Science*, pages 3–22, London, UK, 2015. Springer. doi: 10.1007/978-3-662-46678-0_1.

[45] Frank Pfenning, Luís Caires, and Bernardo Toninho. Proof-carrying code in a session-typed process calculus. In Jean-Pierre Jouannaud and Zhong Shao, editors, *CPP 2011*, volume 7086 of *Lecture Notes in Computer Science*, pages 21–36, Kenting, Taiwan, 2011. Springer. doi: 10.1007/978-3-642-25379-9_4.

[46] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002. ISBN 978-0-262-16209-8.

[47] Diogo Poças, Gil Silva, and Vasco T. Vasconcelos. Simple grammar bisimilarity, with an application to session type equivalence. *CoRR*, abs/2407.04063, 2024. doi: 10.48550/ARXIV.2407.04063. URL https://doi.org/10.48550/arXiv.2407.04063.

[48] Gil Silva, Andreia Mordido, and Vasco T. Vasconcelos. Subtyping context-free session types. In Guillermo A. Pérez and Jean-François Raskin, editors, *34th International Conference on Concurrency Theory, CONCUR 2023, Antwerp, Belgium, September 18-23, 2023*, volume 279 of *LIPIcs*, pages 11:1–11:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023. doi: 10.4230/LIPICS.CONCUR.2023.11.

[49] Janek Spaderna, Peter Thiemann, Andreia Mordido, and Vasco T. Vasconcelos. Parametrized algebraic protocols: Artifact. https://doi.org/10.5281/zenodo.7804667, mar 2023.

[50] Arnaud Spiwack, Csongor Kiss, Jean-Philippe Bernardy, Nicolas Wu, and Richard A. Eisenberg. Linearly qualified types: generic inference for capabilities and uniqueness. *Proc. ACM Program. Lang.*, 6(ICFP):137–164, 2022. doi: 10.1145/3547626.

[51] Kaku Takeuchi, Kohei Honda, and Makoto Kubo. An interaction-based language and its typing system. In Constantine Halatsis, Dimitris G. Maritsas, George Philokyprou, and Sergios Theodoridis, editors, *PARLE '94*, volume 817 of *Lecture Notes in Computer Science*, pages 398–413, Athens, Greece, 1994. Springer. doi: 10.1007/3-540-58184-7_118.

[52] Peter Thiemann and Vasco T. Vasconcelos. Context-free session types. In Jacques Garrigue, Gabriele Keller, and Eijiro Sumii, editors, *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, ICFP 2016*, pages 462–475, Nara, Japan, 2016. ACM. doi: 10.1145/2951913.2951926.

[53] Peter Thiemann and Vasco T. Vasconcelos. Label-dependent session types. *Proc. ACM Program. Lang.*, 4(POPL):67:1–67:29, 2020. doi: 10.1145/3371135.

[54] Bernardo Toninho, Luís Caires, and Frank Pfenning. Higher-order processes, functions, and sessions: A monadic integration. In Matthias Felleisen and Philippa Gardner, editors, *Programming Languages and Systems - 22nd European Symposium on Programming, ESOP 2013*, volume 7792 of *Lecture Notes in Computer Science*, pages 350–369, Rome, Italy, 2013. Springer. doi: 10.1007/978-3-642-37036-6_20.

[55] Bernardo Toninho, Luís Caires, and Frank Pfenning. Corecursion and non-divergence in session-typed processes. In Matteo Maffei and Emilio Tuosto, editors, *Trustworthy Global Computing - 9th International Symposium, TGC 2014*, volume 8902 of *Lecture Notes in Computer Science*, pages 159–175, Rome, Italy, 2014. Springer. doi: 10.1007/978-3-662-45917-1_11.

[56] Vasco T. Vasconcelos. Fundamentals of session types. *Inf. Comput.*, 217:52–70, 2012. doi: 10.1016/j.ic.2012.05.002.

[57] David Walker. *Advanced Topics in Types and Programming Languages*, chapter Substructural Type Systems, pages 3–44. The MIT Press, 2005.

A Frequently Asked Questions

A.1 Session types vs. protocol types

A session type can be assigned to a channel end to describe the entire communication behavior of this channel up to its closing. Thus, a session type classifies a run-time value, namely a channel end. A protocol type does not correspond to any run-time value. It describes composable behavior for bidirectional communication. A protocol type becomes part of a session type by binding it to a particular direction with the ! and ? operators.

In a sense, a protocol type describes a composable part of a communication protocol or the difference between two session types. This view is emphasized by our pervasive use of channel passing style.

A.2 Negated recursion

It is possible to define an algebraic protocol that negates the polarity of its recursive use. Such a protocol is fine, though unusual as it changes direction with every unfolding of the recursion. Here is an example:

```
protocol Flipper = Flipper -Int -Flipper
```

A server for `Flipper` can be implemented with a pair of mutually recursive functions or even with just one function:

```
flipper : !Flipper.End → Unit
flipper c = let c = select Flipper [End] c in
            let (x, c) = receive [Int, ?Flipper.End] c in
            match c with {
              Flipper c → send [Int, !Flipper.End] x c ▷ flipper }
```

A.3 Mutual recursion

Mutually recursive protocols pose no problem. In general, the server would be implemented by mutually recursive functions.

```
protocol Flip = Flip -Int Flop
protocol Flop = Flop Int Flip

flip : !Flip.End → Unit
flip c = select Flip [End] c ▷ receive [Int, !Flop.End] ▷ flop

flop : (Int, !Flop.End) → Unit
flop p = let (x, c) = p in
         select Flop [End] c ▷ send [Int, !Flip.End] x ▷ flip
```

A.4 Duality vs. polarities

The `Dual` operator works on *session types* and it inverts the direction of communication from the outside. Intuitively, the `Dual` operator traverses the spine of a session type and flips all ! to ? and vice versa. It does **not** enter the type of the message "payload". For example (with T the payload type and S the continuation session):

$$\texttt{Dual}\ (?T.S) \equiv\ !T.\texttt{Dual}\ S \qquad\qquad \texttt{Dual}\ (!T.S) \equiv\ ?T.\texttt{Dual}\ S$$

In contrast, the polarity operator - works on *protocol types* and inverts the direction of communication from the inside. This inversion happens at the boundary where a protocol type ρ is turned into a payload type for a session:

$$?(-P).S \equiv !P.S \qquad\qquad !(-P).S \equiv ?P.S$$

The soundness of the type system guarantees that the direction of communication is finally settled by the time we get to a primitive sending or receiving operation.

Like `Dual`, the - operator is involutory:

$$\texttt{Dual (Dual } S) \equiv S \qquad\qquad -(-T) \equiv T$$

A.5 Recursion and duality

Bernardi and Hennessy [10, 11] discovered a problem in the interaction between duality and recursive session types, which is exposed most concisely by the recursive type $T = \mu X.!X.X$. The dual of this type is $\texttt{Dual } (\mu X.!X.X) = \mu X.?(\mu X.!X.X).X$, which can be seen by dualizing the type's expansion

$$\texttt{Dual } (\mu X.!X.X) = \texttt{Dual } (!(\mu X.!X.X).(\mu X.!X.X)) = ?(\mu X.!X.X).\texttt{Dual } (\mu X.!X.X)$$

Using equations we could define the type T by $T = !T.T$. Translated to algebraic session types, the protocol underlying T is as follows:

```
protocol X = Mu T X
type T = !X.End

selectMu : T → !T.T
selectMu c = select Mu [End] c

dualT : Dual T → ?X.End
dualT c = c

matchMu : Dual T → ?T.(Dual T)
matchMu c' = match c' with { Mu c' → c' }
```

This behavior matches exactly the outcome for T according to Bernardi and Hennessy [11]. The function `selectMu` performs the unfolding of T whereas `matchMu` performs the unfolding of `Dual` T. The type of the identity function `dualT` shows that `Dual T = ?X.End`.

A.6 Efficiency of generic servers

In **??**, we explore an example with generic servers to showcase the expressivity of the proposed parametric protocols. The example has a significant overhead as there are many tagging operations compared to a manually crafted version.

The present work is a first step and does not offer a solution supported by formal theorems. But, let us call a protocol ρ *tagless* if it is non-recursive and declares a single constructor tag $C\ T_1 \ldots T_n$. In analogy to Haskell's `newtype`, an implementation can reduce the run-time cost of a tagless protocol to zero by omitting the select and match operations.

A.7 Equirecursive vs. isorecursive session types

Most work on session types is using equirecursive types for two reasons. One is historic: Gay and Hole [22, 23] were the first to study coinductively defined relations on (tail-) recursive session types. They defined the (by now) standard notion of subtyping and showed that it is decidable. Subsequent

work has built on top of their results. The other reason is that "The equirecursive interpretation of a session type guarantees that no message is required for the unfolding of a recursive type." (Balzer and Pfenning [7])

Algebraic protocols conflate the unfolding of the recursion with the branching, just like algebraic datatypes do. For protocols that have finite runs, algebraic protocol do not cause extra messages. As such protocols use recursion guarded by branching, a message to decide the branching is needed anyway.

The protocol `Stream a` in Section 2.4 does cause extra messages as it (roughly) corresponds to an isorecursive reading of the type $\mu X.!a.X$.

A.8 Subtyping, type classes, type inference

Subtyping is undecidable for context-free session types [42]. However, as AlgST's session typing relies mostly on nominal types, there is scope for decidable subtyping. We have a concrete proposal, but it is not implemented and its metatheory has not been checked, yet.

The types `Send a` and `Recv a` for sending and receiving data of type `a` in **??** look like types for a dictionary in a language with type classes or traits [27, 31]. In future work we plan to explore the addition of implicit arguments as in Scala [39].

Type inference is future work.

A.9 Settling of direction

It might be confusing to see a type like $?T.S$ and $!T.S$, but with the possibility that the direction changes later on. However, this behavior is completely predictable. As long as T is a protocol type variable $\alpha : \mathbf{P}$, the direction can change because α could be substituted by some protocol $-T'$. As soon as T is an ordinary type of kind $\mathbf{T}$ (even if T is a variable), the direction is fixed. For example, in the type of primitive send `∀(a:M). ∀(s:S). a → !a.s → s`, the payload type `a` has kind $\mathbf{M}$, so the direction is fixed.

B A Toolbox for Generic Servers

Analogously to the `Stream` protocol, we can define a protocol and a generic server for finite repetion of a subsidiary protocol:

```
protocol Repeat x = More x (Repeat x) | Quit

repeat : ∀(p:P). Service p → Service (Repeat p)
repeat [p] serveP [s] c = match c of {
  Quit c → c,
  More c → serveP [?Repeat p.s] c ▷ repeat [p] serveP [s]
}
```

This protocol may then be used similarly to `Stream`:

```
repeatArith = repeat [Arith] serveArith
```

We can define an active version that offers the subprotocol n times and use that with `Arith`, too:

```
repeatActive : Int → ∀(p:P). Service p → Service -(Repeat -p)
repeatActiveArith : Int → Service -(Repeat -Arith)
```

While the implementation is straightforward, the astute reader might ask why we provide the negative parameter to `Service` in `Service -(Repeat -p)`. In particular, could we not use the `Dual` operator to switch the direction of the service?

To see the problem with this proposal, let's expand the two candidates:

```
Dual (Service (Repeat -p)) = Dual (∀(s:S). ?Repeat -p.s → s)
```

This proposal is doomed because the use of `Dual` on the right side is not well-kinded, as neither the universal type nor the function types are session types in our system.

```
Service -(Repeat -p) = ∀(s:S). ?- (Repeat -p). s → s
                     = ∀(s:S). !Repeat -p. s → s
```

The negative parameter swaps the receiving operator in `Service` to sending with surgical precision.

C Proofs for Types

Example C.1. We generalize the example from the introduction to a parametric stack protocol.

```
protocol Stack a = Pop -a | Push a (Stack a) (Stack a)
```

We set $\Delta = \mathtt{Neg} : \mathbf{P} \longrightarrow \mathbf{P}, \mathtt{a} : \mathbf{P}$ and obtain the straightforward derivation

$$\dfrac{\dfrac{\dfrac{\Delta \vdash \mathtt{a} \Rightarrow \mathbf{P}}{\Delta \vdash \mathtt{a} \Leftarrow \mathbf{P}}}{\Delta \vdash \mathtt{-a} \Leftarrow \mathbf{P}} \qquad \dfrac{\Delta \vdash \mathtt{a} \Rightarrow \mathbf{P}}{\Delta \vdash \mathtt{a} \Leftarrow \mathbf{P}} \qquad \dfrac{\mathtt{Stack} : \mathbf{P} \longrightarrow \mathbf{P} \in \Delta \qquad \dfrac{\Delta \vdash \mathtt{a} \Rightarrow \mathbf{P}}{\Delta \vdash \mathtt{a} \Leftarrow \mathbf{P}}}{\Delta \vdash \mathtt{Stack\ a} \Rightarrow \mathbf{P}}}{\Delta \vdash \mathtt{Stack\ a} \Leftarrow \mathbf{P}}}{\vdash \mathtt{Stack} \Rightarrow \mathbf{P} \longrightarrow \mathbf{P}}$$

The three subderivations correspond to the arguments of the protocol constructors. We conclude that the definition of the protocol `Stack a` is well-formed.

Lemma C.2 (Properties of the materialization and directional operators).

(1) $\Delta \vdash \mathtt{Dual}\ (\S(T).S) \equiv \S(-(T)).\mathtt{Dual}\ S : \boldsymbol{s}$
(2) $-(+(T)) = -(T)$
(3) $-(-(T)) = +(T)$

Proof.

(1) By case analysis on the definition of the § operator (Figure 3).
Case $T = -U$. Since $\S(-U).S = ?U.S$, rule C-DualIn gives $\Delta \vdash \mathtt{Dual}\ (\S(-U).S) \equiv !U.\mathtt{Dual}\ S : \mathbf{s}$. Observing that $-(-U) = +(U) = U$ we conclude that $\S(-(-U)).\mathtt{Dual}\ S = !U.\mathtt{Dual}\ S$.
Case $T \neq -U$ Since $\S(T).S = !T.S$, rule C-DualOut gives $\Delta \vdash \mathtt{Dual}\ (\S(T).S) \equiv ?T.\mathtt{Dual}\ S : \mathbf{s}$.
Conclude observing that $-(T) = -T$.
(2) The proof follows by case analysis on the definition of the directional operators.
Case $T = -U$. We have: $-(+(-U)) = -(-(U)) = -(-U)$.
Case $T \neq -U$. We have: $-(+(T)) = -(T)$.
(3) The proof follows by case analysis on the definition of the directional operators.
Case $T = -U$. We have: $-(-(-U)) = -(+(U)) = -(U) = -U = +(-U)$.
Case $T \neq -U$. We have: $-(-(T)) = -(-T) = +(T)$.

□

Lemma C.3 (Agreement for type conversion). *If* $\Delta \vdash T \equiv U : \kappa$, *then* $\Delta \vdash T \Leftarrow \kappa$ *and* $\Delta \vdash U \Leftarrow \kappa$.

Proof. By rule induction on the hypothesis. □

We start by specifying the syntax of types in normal form. Normal forms for well-formed types are Q as defined by the following grammar:

$$Q ::= R \mid -R$$

$$R ::= \mathtt{Unit} \mid R \to R \mid R \otimes R \mid \forall \alpha\colon \kappa.R \mid \alpha \mid ?R.R \mid !R.R \mid \mathtt{End}_? \mid \mathtt{End}_! \mid \mathtt{Dual}\ \alpha \mid \rho\ \overline{Q}$$

Proposition C.4 (Kind preservation).

(1) *For all* $\Delta \vdash T \Rightarrow \kappa$, $\Delta \vdash \mathrm{nrm}^+(T) \equiv T : \kappa$.
(2) *For all* $\Delta \vdash T \Rightarrow \boldsymbol{s}$, $\Delta \vdash \mathrm{nrm}^-(T) \equiv \mathtt{Dual}\ T : \boldsymbol{s}$.

Proof. See Agda development `nf-sound+` and `nf-sound-`. □

Proof of Theorem 3.1 (Soundness). Item 1. Suppose that $\mathrm{nrm}^+(T) =_\alpha \mathrm{nrm}^+(U)$. From Theorem C.4, we have $\Delta \vdash \mathrm{nrm}^+(T) \equiv T : \kappa$ and $\Delta \vdash \mathrm{nrm}^+(U) \equiv U : \kappa$. Conclude by symmetry and transitivity of conversion as type equality is in $\equiv$. The proof for Item 2 is analogous. □

Proof of Theorem 3.2 (Completeness). See Agda development `nf-complete` and `nf-complete-`. □

To obtain a complexity bound for the computation of a normal form, we exploit its syntactic form.

Lemma C.5. *If* $\mathrm{nrm}^\pm(T) = T'$, *then* T' *is described by* Q.

Proof. See Agda development `nf-normal`. □

Proposition C.6. *For each* $\Delta \vdash T \Rightarrow \kappa$, *the worst case running time of* $\mathrm{nrm}^\pm(T)$ *is in* $O(|T|)$.

Proof. A run of $\mathrm{nrm}^\pm(T)$ visits each of the $|T|$ nodes in the input type once. At each node, it either reconstructs the type from the normal forms of the sub-terms in $O(1)$ time, or (in the transmission rules) it applies polarity correction to normal forms. By Lemma C.5, a normal form starts with at most one negative polarity, so correction takes at most two steps, at cost $O(1)$. □

Proof of Theorem 3.3 (Time complexity of type equivalence). A corollary of Theorem C.5. □

D Proofs for expressions and processes

Lemma D.1.

(1) *If* $\Delta \mid \Gamma_1 \vdash e \Rightarrow T \mid \Gamma_2$ *and* $\Delta \vdash \Gamma_1 \Leftarrow \boldsymbol{\tau}$, *then* $\Delta \vdash \Gamma_2 \Leftarrow \boldsymbol{\tau}$ *and* T *is in normal form.*
(2) *If* $\Delta \mid \Gamma_1 \vdash e \Leftarrow T \mid \Gamma_2$ *and* $\Delta \vdash \Gamma_1 \Leftarrow \boldsymbol{\tau}$ *and* T *is in normal form, then* $\Delta \vdash \Gamma_2 \Leftarrow \boldsymbol{\tau}$.

Proof.

(1) By rule induction on the first hypothesis.
(2) Using Item 1 and observing that a type U such that $U \leq T$ is also in normal form.

□

Lemma D.2 (Properties of normalization).

(1) *If* $\Delta \vdash \mathrm{nrm}^+(T) \Rightarrow \kappa$, *then* $\Delta \vdash T \Leftarrow \kappa$.
(2) *If* $\Delta \vdash \mathrm{nrm}^-(T) \Rightarrow \kappa$, *then* $\Delta \vdash T \Leftarrow \boldsymbol{s}$.
(3) *If* $\Delta \vdash \S(T).S \Rightarrow \kappa$, *then* $\Delta \vdash T \Leftarrow \boldsymbol{p}$ *and* $\Delta \vdash S \Leftarrow \boldsymbol{s}$.
(4) *If* $\Delta \vdash \pm(T) \Rightarrow \kappa$, *then* $\Delta \vdash T \Leftarrow \kappa$.

Labelled transition system for expressions $\boxed{e \xrightarrow{\lambda} e}$

Act-App $(\lambda x\colon T.e)\ v \xrightarrow{\beta} e[v/x]$

Act-TApp $(\Lambda\,\alpha\colon\kappa.v)[T] \xrightarrow{\beta} v[T/\alpha]$

Act-Let $\mathtt{let}\ \langle x, y\rangle = \langle u, v\rangle\ \mathtt{in}\ e \xrightarrow{\beta} e[u/x][v/y]$

Act-Let* $\mathtt{let} * = *\ \mathtt{in}\ e \xrightarrow{\beta} e$

Act-Rec $(\mathtt{rec}\ x\colon T.v)\ u \xrightarrow{\beta} (v[\mathtt{rec}\ x\colon T.v/x])\ u$

Act-Fork $\mathtt{fork}\ v \xrightarrow{\mathtt{fork}\ v} *$

Act-New $\mathtt{new}[T] \xrightarrow{(vx\,y)\,T} \langle x, y\rangle$

Act-Rcv $\mathtt{receive}[T][U]\ x \xrightarrow{x?v} \langle v, x\rangle$

Act-Send $\mathtt{send}[T][U]\ v\ x \xrightarrow{x!v} x$

Act-Match $\mathtt{match}\ x\ \mathtt{with}\ \{C_i\ y_i \to e_i\}_{i\in I} \xrightarrow{x?C_k} e_k[x/y_k]$

Act-Sel $\mathtt{select}\ C[\overline{T}]\ x \xrightarrow{x!C} x$

Act-Wait $\mathtt{wait}\ x \xrightarrow{x?} *$

Act-Term $\mathtt{terminate}\ x \xrightarrow{x?} *$

Act-AppL $\dfrac{e_1 \xrightarrow{\lambda} e_2}{e_1\ e_3 \xrightarrow{\lambda} e_2\ e_3}$

Act-AppR $\dfrac{e_1 \xrightarrow{\lambda} e_2}{v\ e_1 \xrightarrow{\lambda} v\ e_2}$

Act-TAppE $\dfrac{e_1 \xrightarrow{\lambda} e_2}{e_1[T] \xrightarrow{\lambda} e_2[T]}$

Act-PairL $\dfrac{e_1 \xrightarrow{\lambda} e_2}{\langle e_1, e_3\rangle \xrightarrow{\lambda} \langle e_2, e_3\rangle}$

Act-PairR $\dfrac{e_1 \xrightarrow{\lambda} e_2}{\langle v, e_1\rangle \xrightarrow{\lambda} \langle v, e_2\rangle}$

Act-MatchE $\dfrac{e_1 \xrightarrow{\lambda} e_2}{\mathtt{match}\ e_1\ \mathtt{with}\ \{C_i\ x_i \to e_i\}_{i\in I} \xrightarrow{\lambda} \mathtt{match}\ e_2\ \mathtt{with}\ \{C_i\ x_i \to e_i\}_{i\in I}}$

Act-LetE $\dfrac{e_1 \xrightarrow{\lambda} e_2}{\mathtt{let}\ \langle x, y\rangle = e_1\ \mathtt{in}\ e_3 \xrightarrow{\lambda} \mathtt{let}\ \langle x, y\rangle = e_2\ \mathtt{in}\ e_3}$

Act-LetE* $\dfrac{e_1 \xrightarrow{\lambda} e_2}{\mathtt{let} * = e_1\ \mathtt{in}\ e_3 \xrightarrow{\lambda} \mathtt{let} * = e_2\ \mathtt{in}\ e_3}$

Fig. 12. Labelled transition system for expressions

Proof. The proof is by mutual induction on the various hypotheses. Most cases follow by a straightforward induction. We detail the case for $\mathrm{nrm}^-(?T.S) = \S(+(\mathrm{nrm}^+(T))).\mathrm{nrm}^-(S)$. Item 3 gives $\Delta \vdash +(\mathrm{nrm}^+(T)) \Leftarrow \mathbf{P}$ and $\Delta \vdash \mathrm{nrm}^-(S) \Leftarrow \mathbf{S}$. Rule T-Sub gives $\Delta \vdash +(\mathrm{nrm}^+(T)) \Rightarrow \mathbf{P}$ and $\Delta \vdash \mathrm{nrm}^-(S) \Rightarrow \mathbf{S}$. Item 4 gives $\Delta \vdash \mathrm{nrm}^+(T) \Rightarrow \mathbf{P}$. Again rule T-Sub gives $\Delta \vdash \mathrm{nrm}^+(T) \Leftarrow \mathbf{P}$. Item 1 gives $\Delta \vdash T \Rightarrow \mathbf{P}$. Once again, rule T-Sub gives $\Delta \vdash T \Leftarrow \mathbf{P}$. From $\Delta \vdash \mathrm{nrm}^-(S) \Leftarrow \mathbf{S}$, induction gives $\Delta \vdash S \Rightarrow \mathbf{S}$ and T-Sub gives $\Delta \vdash S \Leftarrow \mathbf{S}$. Finally, rule T-In gives $\Delta \vdash ?T.S \Rightarrow \mathbf{S}$ and rule T-Sub yields $\Delta \vdash ?T.S \Leftarrow \mathbf{S}$. □

Lemma D.3 (Substitution).

(1) *If* $\Delta, \alpha\colon\kappa \vdash T \Rightarrow \kappa'$ *and* $\Delta \vdash U \Leftarrow \kappa$, *then* $\Delta \vdash T[U/\alpha] \Rightarrow \kappa'$.
(2) *If* $\Delta, \alpha\colon\kappa \mid \Gamma_1 \vdash e \Rightarrow T \mid \Gamma_2$ *and* $\Delta \vdash U \Leftarrow \kappa$, *then* $\Delta \mid \Gamma_1 \vdash e[U/\alpha] \Rightarrow \mathrm{nrm}^+(T[U/\alpha]) \mid \Gamma_2$.
(3) *If* $\Delta \mid \Gamma_1, x\colon U \vdash e \Rightarrow T \mid \Gamma_2$ *and* $\Delta \mid \Gamma_3 \vdash v \Leftarrow U \mid \cdot$, *then* $\Delta \mid \Gamma_1, \Gamma_3 \vdash e[v/x] \Rightarrow T \mid \Gamma_2$.
(4) *If* $\Delta \mid \Gamma_1, x\colon^{\star} U \vdash e \Rightarrow T \mid \Gamma_2, x\colon^{\star} U$ *and* $\Delta \mid \Gamma_3 \vdash v \Leftarrow U \mid \cdot$, *then* $\Delta \mid \Gamma_1, \Gamma_3 \vdash e[v/x] \Rightarrow T \mid \Gamma_2$.

Proof. The proof follows by mutual induction on the first hypothesis. □

Lemma D.4 (Agreement). *Let* $\Delta \vdash \Gamma_1 \Leftarrow \mathbf{T}$.

(1) *If* $\Delta \mid \Gamma_1, x\colon T \vdash e \Rightarrow U \mid \Gamma_2$, *then* $\Delta \vdash T \Leftarrow \kappa$.

Labelled transition system for contexts

$$\boxed{\Gamma \xrightarrow{\lambda} \Gamma \quad \Gamma \xrightarrow{\pi} \Gamma}$$

L-Rcv
$$\frac{\cdot \mid \Gamma \vdash v \Leftarrow T \mid \cdot}{x\colon ?T.S \xrightarrow{x?v} \Gamma, x\colon S}$$

L-Send
$$\frac{\cdot \mid \Gamma \vdash v \Leftarrow T \mid \cdot}{\Gamma, x\colon !T.S \xrightarrow{x!v} x\colon S}$$

L-Match
$$\frac{\texttt{protocol}\ \rho\ \overline{\alpha} = \{C_i\ \overline{T_i}\}_{i\in I}}{x\colon ?(\rho\ \overline{U}).S \xrightarrow{x?C_k} x\colon \S(-(\overline{T_k}[\overline{U}/\overline{\alpha}])).S}$$

L-Sel
$$\frac{\texttt{protocol}\ \rho\ \overline{\alpha} = \{C_i\ \overline{T_i}\}_{i\in I}}{x\colon !(\rho\ \overline{U}).S' \xrightarrow{x!C_k} x\colon \S(+(\overline{T_k}[\overline{U}/\overline{\alpha}])).S'}$$

L-Wait
$$x\colon \mathsf{End}_? \xrightarrow{x?} \cdot$$

L-Term
$$x\colon \mathsf{End}_! \xrightarrow{x!} \cdot$$

L-Beta
$$\cdot \xrightarrow{\beta} \cdot$$

L-Fork
$$\frac{\cdot \mid \Gamma \vdash v \Leftarrow \mathsf{Unit} \to \mathsf{Unit} \mid \cdot}{\Gamma \xrightarrow{\mathsf{fork}\, v} \cdot}$$

L-New
$$\cdot \xrightarrow{(\nu x y)\, T} x\colon \mathrm{nrm}^+(T), y\colon \mathrm{nrm}^-(T)$$

L-Tau
$$\cdot \xrightarrow{\tau} \cdot$$

L-OpenL
$$x\colon !T.S \xrightarrow{(vab)x!a} b\colon \mathrm{nrm}^-(T), x\colon S$$

L-OpenR
$$x\colon !T.S \xrightarrow{(vab)x!b} a\colon \mathrm{nrm}^-(T), x\colon S$$

L-Par
$$\frac{\Gamma_1 \xrightarrow{\pi_1} \Gamma_3 \qquad \Gamma_2 \xrightarrow{\pi_2} \Gamma_4}{\Gamma_1, \Gamma_2 \xrightarrow{\pi_1 \mid \pi_2} \Gamma_3, \Gamma_4}$$

Fig. 13. Labelled transition system for typing contexts

(2) *If* $\Delta \mid \Gamma_1 \vdash e \Rightarrow T \mid \Gamma_2$, *then* $\Delta \vdash T \Leftarrow \kappa$.
(3) *If* $\Gamma_1, x\colon T \vdash p$, *then* $\cdot \vdash T \Leftarrow \kappa$.

Proof.

(1) By rule induction on the first hypothesis.
(2) By rule induction on the first hypothesis, using Theorem D.3 and Theorem C.3.
(3) By rule induction on the first hypothesis, using E-Check and Item 1.

□

Monotonicity tells us that resources are not created during the process of type checking.

Lemma D.5 (Monotonicity). *If* $\Delta \mid \Gamma_1 \vdash v \Rightarrow T \mid \Gamma_2$, *then* $\Gamma_2 \subseteq \Gamma_1$.

Proof. A straightforward rule induction on the hypothesis. The base cases are E-Const and E-Var. The cases for E-Abs, E-TAbs, E-Pair and E-TApp follow by induction and transitivity of the subset relation. □

Lemma D.6 (Strengthening).

(1) *If* $\Delta, \alpha\colon \kappa \vdash T \Rightarrow \kappa'$ *and* $\alpha \notin \mathrm{fv}(T)$, *then* $\Delta \vdash T \Rightarrow \kappa'$.
(2) *If* $\Delta \mid \Gamma_1, x\colon U \vdash e \Rightarrow T \mid \Gamma_2, x\colon U$, *then* $\Delta \mid \Gamma_1 \vdash e \Rightarrow T \mid \Gamma_2$.

Proof.

(1) The proof is by induction hypothesis on the first hypothesis, using T-Sub.
(2) The proof is by induction hypothesis on the first hypothesis.

□

Lemma D.7 (Weakening). *If* $\Delta \mid \Gamma_1 \vdash e \Rightarrow T \mid \Gamma_2$, *then* $\Delta \mid \Gamma_1, x\colon U \vdash e \Rightarrow T \mid \Gamma_2, x\colon U$.

Proof. The proof follows by induction on the hypothesis. □

Preservation and progress. We now prove the main results.

Proof of Theorem 4.1 (Preservation).
The proof follows by mutual rule induction on the first hypothesis.

(1) Rule Act-App. In this case, $(\lambda x\colon U.e)\, v \xrightarrow{\beta} e[v/x]$ and $\Gamma_2 = \Gamma_2' = \cdot$. Inversion of E-App on $\Delta \mid \Gamma_1, \Gamma_3 \vdash (\lambda x\colon U.e)\, v \Rightarrow T \mid \Gamma_3$ yields (a) $\Delta \mid \Gamma_1, \Gamma_3 \vdash \lambda x\colon U.e \Rightarrow V \to T \mid \Gamma'$ and (b) $\Delta \mid \Gamma' \vdash v \Leftarrow V \mid \Gamma_3$. By monotonicity (Theorem D.5), we know that $\Gamma_3 \subseteq \Gamma'$; say that $\Gamma' = \Gamma_1', \Gamma_3$. Applying inversion of E-Abs on (a) and strengthening (Theorem D.6) on (b), we get $\Delta \mid \Gamma_1, \Gamma_3, x\colon \mathrm{nrm}^+(U) \vdash e \Rightarrow T \mid \Gamma_1', \Gamma_3$ and $\Delta \mid \Gamma_1' \vdash v \Leftarrow V \mid \cdot$ (and $V = \mathrm{nrm}^+(U)$). By C-Refl and agreement (Theorem D.4) on the second hypothesis, we know that $T \equiv_A T$. Conclude applying the substituion lemma (Theorem D.3, Item 3) and weakening (Theorem D.7).
Rule Act-TApp. Again, $\Gamma_2 = \Gamma_2' = \cdot$. Apply inversion of E-TApp on the second hypothesis, followed by inversion of E-TAbs and conclude with Theorem D.3, Item 2, using $T' = T$.
Rule Act-Let. Apply inversion of E-Let to the second hypothesis, followed by inversion of E-Pair, then apply substitution (Theorem D.3, Item 3) and conclude using $T' = T$.
Rule Act-Let*. In this case, $\mathtt{let}\ * = *\ \mathtt{in}\ e \xrightarrow{\beta} e$ and $\Gamma_2 = \Gamma_2' = \cdot$. Using inversion of E-Const and E-Check for $*$, we know that premises to E-Let* include $\Delta \mid \Gamma_1, \Gamma_3 \vdash e \Rightarrow T \mid \Gamma_3$, which is what we want to prove, since $T \leq T$.
Rule Act-Rec. Again, $\Gamma_2 = \Gamma_2' = \cdot$. Premises to rule E-App are $\Delta \mid \Gamma_1, \Gamma_3 \vdash \mathtt{rec}\, x\colon V.v \Rightarrow U \to T \mid \Gamma_1, \Gamma_3$ and $\Delta \mid \Gamma_1, \Gamma_3 \vdash u \Leftarrow U \mid \Gamma_3$. Applying inversion of E-Rec on the former, conclude that $\mathrm{nrm}^+(V) = U \to T$ and then apply inversion of E-Check and the substitution lemma (Theorem D.3) to get $\Delta \mid \Gamma_1, \Gamma_3 \vdash v[\mathtt{rec}\, x\colon V.v/x] \Rightarrow U \to T \mid \Gamma_1, \Gamma_3$. Conclude applying E-App and using the fact that $T \leq T$.
Rule Act-Fork. Inversion of E-App yields $\Delta \mid \Gamma_1, \Gamma_2, \Gamma_3 \vdash \mathtt{fork} \Rightarrow U \to T \mid \Gamma'$ and $\Delta \mid \Gamma' \vdash v \Leftarrow U \mid \Gamma_3$. Applying E-Const on the former we know that $\Gamma' = \Gamma_1, \Gamma_2, \Gamma_3$, $U = \mathtt{Unit} \to \mathtt{Unit}$ and $T = \mathtt{Unit}$. By L-Fork we know that $\Delta \mid \Gamma_2 \vdash v \Leftarrow \mathtt{Unit} \to \mathtt{Unit} \mid \cdot$ and $\Gamma_1 = \cdot$ in this case. By applying E-Const for $*$ we conclude that $\Delta \mid \Gamma_3 \vdash * \Rightarrow \mathtt{Unit} \mid \Gamma_3$, as we wanted.
Rule Act-New. In this case, $\mathtt{new}[U] \xrightarrow{(\nu x\, y)\, U} \langle x, y\rangle$. By L-New, we know that $\Gamma_2 = \cdot$ and $\Gamma_2' = x\colon \mathrm{nrm}^+(U), y\colon \mathrm{nrm}^-(U)$. Inversion of E-TApp and E-Const on the second hypothesis yields $T = \mathrm{nrm}^+(U) \otimes \mathrm{nrm}^-(U)$ and $\Gamma_1 = \cdot$. Using E-Var and E-Pair we conclude that $\Delta \mid \Gamma_3, x\colon \mathrm{nrm}^+(U), y\colon \mathrm{nrm}^-(U) \vdash \langle x, y\rangle \Rightarrow \mathrm{nrm}^+(U) \otimes \mathrm{nrm}^-(U) \mid \Gamma_3$.
Rule Act-Rcv. In this case, $\mathtt{receive}[U][V]\, x \xrightarrow{x?v} \langle v, x\rangle$. By L-Rcv, $\Gamma_2 = x\colon ?U.S$ and $\Gamma_2' = x\colon S, \Gamma$, where Γ is such that $\cdot \mid \Gamma \vdash v \Leftarrow U \mid \cdot$. Inversion of E-App on the second hypothesis yields $\Delta \mid \Gamma_1, x\colon ?U.S, \Gamma_3 \vdash \mathtt{receive}[U][V] \Rightarrow W \to T \mid \Gamma_1'$ and $\Delta \mid \Gamma_1' \vdash x \Leftarrow W \mid \Gamma_3$. Inversion of E-Var on the latter yields $\Gamma_1' = x\colon ?U.S, \Gamma_3$ and $W = ?U.S$. Inversion of E-TApp applied twice, followed by E-Const enables us to conclude that $\Gamma_1 = \cdot$ and $T = U \otimes S$. Using L-Rcv, E-Check and weakening we know that $\Delta \mid x\colon S, \Gamma, \Gamma_3 \vdash v \Rightarrow U \mid x\colon S, \Gamma_3$, using E-Var and weakening we get $\Delta \mid x\colon S, \Gamma_3 \vdash x \Rightarrow S \mid \Gamma_3$. Conclude applying E-Pair and recalling that $U \otimes S \leq T$.
Rule Act-Send. In this case, $\mathtt{send}[U][S]\, v\, x \xrightarrow{x!v} x$. By L-Send, $\Gamma_2 = \Gamma, x\colon !U.S$ where Γ is such that $\cdot \mid \Gamma \vdash v \Leftarrow U \mid \cdot$. Proceed similarly to the previous case, applying inversion of E-App twice, followed by E-TApp and then inversion of E-Const over send to conclude that $S = T$. On the other hand, invert E-Var and L-Send with weakening applied to the other premises to conclude that $\Gamma_1 = \cdot$. The result follows applying E-Var and weakening with $\Gamma_2' = x\colon T$, and knowing that $T \leq T$.

Rule Act-Match. In this case, $\texttt{match}\ x\ \texttt{with}\ \{C_i\ y_i \to e_i\}_{i\in I} \xrightarrow{x?C_k} e_k[x/y_k]$. By L-Match, we have $\Gamma_2 = x\colon ?(P\ \overline{U}).S'$ for $\texttt{protocol}\ \rho\ \overline{\alpha} = \{C_i\ \overline{T_i}\}_{i\in I}$. Premises to E-Match include (a) $\Delta \mid \Gamma_1, x\colon ?(P\ \overline{U}).S', \Gamma_3 \vdash x \Rightarrow ?(P\ \overline{U}).S' \mid \Gamma_1, \Gamma_3$, (b) $\Delta \mid \Gamma_1, \Gamma_3, y_i\colon \S(-(\overline{T_i}[\overline{U}/\overline{\alpha}])).S \vdash e_i \Rightarrow V_i \mid \Gamma_i$ and (c) $V = \bigsqcup_{i\in I} V_i$. By L-Match we know that $\Gamma_2' = x\colon \S(-(\overline{T_k}[\overline{U}/\overline{\alpha}])).S$. Applying E-Var and E-Check we get (d) $\Delta \mid x\colon \S(-(\overline{T_k}[\overline{U}/\overline{\alpha}])).S \vdash x \Leftarrow \S(-(\overline{T_k}[\overline{U}/\overline{\alpha}])).S \mid \cdot$. Conclude applying the substitution lemma (Theorem D.3) to (b) and (d), and using E-Check together with the definition of the least upper bound.

Rule Act-Sel. We have $\texttt{select}\ C[\overline{U}]\ x \xrightarrow{x!C} x$. Using L-Sel, we know that $\Gamma_2 = x\colon !(P\ \overline{U}).S'$ for $\texttt{protocol}\ \rho\ \overline{\alpha} = \{C_i\ \overline{T_i}\}_{i\in I}$. Use inversion of E-App and E-TApp on the second hypothesis to conclude that $T = \S(+(\overline{T_k}[\overline{U}/\overline{\alpha}])).S'$. Using inversion of E-Var conclude that $\Gamma_1 = \cdot$. Finally, use E-Var and weakening (Theorem D.7) to get $\Delta \mid x\colon \S(+(\overline{T_k}[\overline{U}/\overline{\alpha}])).S', \Gamma_3 \vdash x \Rightarrow \S(+(\overline{T_k}[\overline{U}/\overline{\alpha}])).S' \mid \Gamma_3$. Conclude observing that $\S(+(\overline{T_k}[\overline{U}/\overline{\alpha}])).S' = T \equiv_A T$.

Rule Act-Wait. In this case, $\Gamma_2 = x\colon \texttt{End}_?$ and $\Gamma_2' = \cdot$. Inversion of E-App on the second hypothesis yields $\Delta \mid \Gamma_1, x\colon \texttt{End}_?, \Gamma_3 \vdash \texttt{wait} \Rightarrow U \to T \mid \Gamma'$ and $\Delta \mid \Gamma' \vdash x \Leftarrow U \mid \Gamma_3$. Apply E-Const to the former and E-Var to the latter to conclude that $\Gamma_1 = \cdot$, $U = \texttt{End}_?$ and $T = \texttt{Unit}$. The result follows using E-Const on $*$ and weakening (Theorem D.7), using $T' = T$.

Rule Act-Term is similar.

Rule Act-AppR. In this case, $v\ e_1 \xrightarrow{\lambda} v\ e_2$. Using the second hypothesis and rule E-App we know that (a) $\Delta \mid \Gamma_1, \Gamma_2, \Gamma_3 \vdash v \Rightarrow U \to T \mid \Gamma_1', \Gamma_2, \Gamma_3$ and (b) $\Delta \mid \Gamma_1', \Gamma_2, \Gamma_3 \vdash e_1 \Leftarrow U \mid \Gamma_3$. By induction hypothesis on Item 2, using (b), we get $\Delta \mid \Gamma_1', \Gamma_2', \Gamma_3 \vdash e_2 \Leftarrow U \mid \Gamma_3$. Applying weakening and strengthening to (a) we get $\Delta \mid \Gamma_1, \Gamma_2', \Gamma_3 \vdash v \Rightarrow U \to T \mid \Gamma_1', \Gamma_2', \Gamma_3$ Apply E-App to conclude that $\Delta \mid \Gamma_1, \Gamma_2', \Gamma_3 \vdash v\ e_2 \Rightarrow T \mid \Gamma_3$.

Rules Act-AppL, Act-TAppE, Act-LetE, Act-LetE*, Act-PairL, Act-PairR, Act-MatchE are similar.

(2) Inversion of E-Check on the second hypothesis yields $\Delta \mid \Gamma_1, \Gamma_2, \Gamma_3 \vdash e_1 \Rightarrow U \mid \Gamma_3$ and $U \le T$. By induction hypothesis on Item 1 we know that $\Delta \mid \Gamma_1, \Gamma_2', \Gamma_3 \vdash e_2 \Rightarrow U' \mid \Gamma_3$ with $U' \le U$. By transitivity of subtyping we know that $U' \le T$. Using E-Check we conclude that $\Delta \mid \Gamma_1, \Gamma_2', \Gamma_3 \vdash e_2 \Leftarrow T \mid \Gamma_3$.

(3) Rule Act-Session. Inversion of Act-Session yields $e_1 \xrightarrow{\sigma} e_2$ and inversion of P-Exp on the second hypothesis implies $\cdot \mid \Gamma_1, \Gamma_2 \vdash e_1 \Leftarrow \texttt{Unit} \mid \cdot$. Applying induction hypothesis on Item 2 we get $\cdot \mid \Gamma_1, \Gamma_2' \vdash e_2 \Leftarrow \texttt{Unit} \mid \cdot$. Conclude applying P-Exp.

Rule Act-Beta is similar, considering $\Gamma_2 = \Gamma_2' = \cdot$, as prescribed by L-Tau and L-Beta.

Rule Act-Fork. Inversion of Act-Fork on the first hypothesis and of P-Exp on the second hypothesis yields (a) $e_1 \xrightarrow{\texttt{fork}\ v} e_2$ and (b) $\cdot \mid \Gamma_1 \vdash e_1 \Leftarrow \texttt{Unit} \mid \cdot$. By L-Fork we know that (c) $\Gamma_1 \xrightarrow{\texttt{fork}\ v} \cdot$ and (d) $\cdot \mid \Gamma_1 \vdash v \Leftarrow \texttt{Unit} \to \texttt{Unit} \mid \cdot$. Applying induction hypothesis on Item 2 to (a), (b) and (c) we get $\cdot \mid \cdot \vdash e_2 \Leftarrow \texttt{Unit} \mid \cdot$. Inversion of E-Check on (d) and E-Const for $*$ enables the application of E-App to deduce $\cdot \mid \Gamma_1 \vdash v * \Rightarrow \texttt{Unit} \mid \cdot$. Conclude using E-Check, P-Exp and P-Par.

Rule Act-New. By L-Tau, $\Gamma_2 = \cdot$. Inversion of P-Exp on the second hypothesis yields $\cdot \mid \Gamma_1 \vdash e_1 \Leftarrow \texttt{Unit} \mid \cdot$. Applying inversion of Act-New and induction hypothesis on Item 2 we get $\cdot \mid \Gamma_1, x\colon \mathrm{nrm}^+(T), y\colon \mathrm{nrm}^-(T) \vdash e_2 \Leftarrow \texttt{Unit} \mid \cdot$. Using P-Exp we know that $\Gamma_1, x\colon \mathrm{nrm}^+(T), y\colon \mathrm{nrm}^-(T) \vdash \langle e_2 \rangle$. Conclude using P-New.

Rule Act-JoinL. Inversion of P-Par dictates a context split: $\Gamma_1, \Gamma_2^1 \vdash p_1$ and $\Gamma_1, \Gamma_2^2 \vdash p_2$. Using inversion of L-Par and applying induction hypothesis on Item 3 we get $\Gamma_1, \Gamma_3 \vdash q_1$ and $\Gamma_1, \Gamma_4 \vdash q_2$, where $\Gamma_2' = \Gamma_3, \Gamma_4$. Conclude with P-Par and L-Par.

Rule Act-JoinR is similar.
Rule Act-Msg. In this case, $(\nu xy)\, p_1 \xrightarrow{\tau} (\nu xy)\, p_2$. By inversion of Act-Msg, we know that $p_1 \xrightarrow{x?v \mid y!v} p_2$. For a τ-transition, the second hypothesis reads as $\Gamma_1 \vdash (\nu xy)\, p_1$ (recall rule L-Tau). By P-New, we have $\Gamma, x\colon \mathrm{nrm}^+(T), y\colon \mathrm{nrm}^-(T) \vdash p_1$. By L-Par, L-Rcv and L-Send, we know that $\mathrm{nrm}^+(T) = ?U.S$ and $\Gamma_1, x\colon ?U.S, y\colon !U.\mathrm{nrm}^-(S) \xrightarrow{x?v \mid y!v} \Gamma_1, x\colon S, y\colon \mathrm{nrm}^-(S)$ and $\cdot \mid \Gamma_1 \vdash v \Leftarrow U \mid \cdot$. Applying induction hypothesis on Item 3 we get $\Gamma_1, x\colon S, y\colon \mathrm{nrm}^-(S) \vdash p_2$. Conclude applying P-New.
Rule Act-Bra. Premises to P-New include (a) $\Gamma_1, x\colon \mathrm{nrm}^+(T), y\colon \mathrm{nrm}^-(T) \vdash p_1$. By L-Match, L-Sel and L-Par we know that $\mathrm{nrm}^+(T) = ?(P\ \overline{U}).S'$ where $\mathtt{protocol}\ \rho\ \overline{\alpha} = \{C_i\ \overline{T_i}\}_{i\in I}$ and $C = C_k$ for some $k \in I$. Thus, we can rewrite (a) as $\Gamma_1, x\colon ?(P\ \overline{U}).S', y\colon !(P\ \overline{U}).\mathrm{nrm}^-(S') \vdash p_1$. Inversion of Act-Bra yields $p_1 \xrightarrow{x?C \mid y!C} p_2$. Applying induction hypothesis on Item 3 we get $\Gamma_1, x\colon \S(-(\overline{T_k}[\overline{U}/\overline{\alpha}])).S, y\colon \S(+(\overline{T_k}[\overline{U}/\overline{\alpha}])).\mathrm{nrm}^-(S') \vdash p_2$. Apply Theorem C.2, Item 1 and P-New to conclude.
Rule Act-Wait. Premises to P-New include $\Gamma_1, x\colon \mathrm{nrm}^+(T), y\colon \mathrm{nrm}^-(T) \vdash p_1$. By L-Par, L-Wait and L-Term, we know that $\mathrm{nrm}^+(T) = \mathtt{End}_?$. Applying induction hypothesis on Item 3 we get $\Gamma_1 \vdash p_2$.
Rule Act-ParL. Inversion of Act-ParL yields $p_1 \xrightarrow{\pi} p_2$. Premises to P-Par are (a) $\Gamma_1, \Gamma_2^1 \vdash p_1$ and (b) $\Gamma_1, \Gamma_2^2 \vdash q_1$, where $\Gamma_2 = \Gamma_2^1, \Gamma_2^2$. Applying induction hypothesis on Item 3 to (a) we get $\Gamma_1, \Gamma_3 \vdash p_2$, where $\Gamma_2^1 \xrightarrow{\pi} \Gamma_3$. Conclude using (b) and applying P-Par.
Rule Act-ParR is similar.
Rule Act-Res. Premises to Act-Res are $p_1 \xrightarrow{\pi} p_2$ and $x, y \notin \mathrm{fv}(\pi)$. Since $x, y \notin \mathrm{fv}(\pi)$, we know that $\cdot \xrightarrow{\pi} \cdot$. Premises to P-New include $\Gamma_1, x\colon \mathrm{nrm}^+(T), y\colon \mathrm{nrm}^-(T) \vdash p_1$. Induction hypothesis on Item 3 yields $\Gamma_1, x\colon \mathrm{nrm}^+(T), y\colon \mathrm{nrm}^-(T) \vdash p_2$. Conclude using P-New.
Rule Act-OpenL. By L-OpenL we know that $\Gamma_2 = x\colon !T.S$. Inversion of P-New yields $\Gamma_1, x\colon !T.S, a\colon \mathrm{nrm}^+(U), b\colon \mathrm{nrm}^-(U) \vdash p_1$. Inversion of L-Send, E-Check and E-Var yields $\mathrm{nrm}^+(U) = T$. By induction hypothesis on Item 3, conclude that $\Gamma_1, x\colon S, b\colon \mathrm{nrm}^-(U) \vdash p_2$.
Rule Act-OpenR is similar.
Rule Act-CloseL. By L-Tau, $\cdot \xrightarrow{\tau} \cdot$. Inversion of P-New yields $\Gamma_1, x\colon \mathrm{nrm}^+(T), y\colon \mathrm{nrm}^-(T) \vdash p_1$. By L-Rcv, we know that $\mathrm{nrm}^+(T) = ?U.S$. By L-Rcv, L-OpenL, L-Par and induction hypothesis on Item 3 we get $\Gamma_1, x\colon S, y\colon \mathrm{nrm}^-(S), b\colon \mathrm{nrm}^-(U), \Gamma \vdash p_2$ where $\cdot \mid \Gamma \vdash a \Leftarrow U \mid \cdot$. Applying E-Check and E-Var we know that $\Gamma = a\colon U$. Conclude applying P-New twice.
Rule Act-CloseR is similar.

□

Canonical forms tell us the form of values from the form of types.

Lemma D.8 (Canonical forms). *Let* $\Delta \mid \Gamma_1 \vdash v \Rightarrow T \mid \Gamma_2$.

(1) *If* $T = \mathtt{Unit}$, *then* $v = *$.
(2) *If* $T = U \to V$, *then* $v = \lambda x\colon U.e$ *or* $v = \mathtt{rec}\ x\colon T.e$ *or* $v = \mathtt{fork}$ *or* $v = \mathtt{wait}$ *or* $v = \mathtt{terminate}$ *or* $v = \mathtt{receive}[W][X]$ *or* $v = \mathtt{send}[W][X]$ *or* $v = \mathtt{send}[W][X]\ u$ *or* $v = \mathtt{select}\ C[\overline{W}]$.
(3) *If* $T = U \otimes V$, *then* $v = \langle u, w\rangle$.
(4) *If* $T = \forall \alpha\colon \kappa.U$, *then* $v = \Lambda\, \alpha\colon \kappa.u$ *or* $v = \mathtt{receive}$ *or* $v = \mathtt{receive}[V]$ *or* $v = \mathtt{send}$ *or* $v = \mathtt{send}[V]$ *or* $v = \mathtt{select}\ C[\overline{V}]$ *or* $v = \mathtt{select}\ C$ *or* $v = \mathtt{new}$.
(5) *If* $\Delta \vdash T \Leftarrow \mathbf{s}$, *then* $v = x$.

Proof. A straightforward analysis of the rules that apply to judgement $\Delta \mid \Gamma_1 \vdash v \Rightarrow T \mid \Gamma_2$. The cases for $T = U \to V$ and $T = \forall \alpha\colon \kappa.U$ come from rules E-Abs, E-TAbs as well as from the

constants that bear these two types. The only values for session types (types of kind $\mathbf{s}$) are variables (representing channel ends). □

Lemma D.9 (Completed processes are closed). *If p is completed, then $\cdot \vdash p$.*

Proof of Theorem 4.2 (Progress). By mutual rule induction in the first hypothesis, in each case.

Rules E-Const, E-Var, E-Abs and E-TAbs. Constants, variables, lambda abstractions and type abstractions are values.

Rule E-Let*. Since $e = \texttt{let} * = e_1 \texttt{ in } e_2$ is not a value it must reduce. Distinguish two cases. When e_1 is a value, canonical forms (Theorem D.8) give $e_1 = *$ and e has a transition by rule Act-Let*. Otherwise, the premises to rule E-Let* include $\Delta \mid \Gamma_1^{\mathbf{s}} \vdash e_1 \Leftarrow \texttt{Unit} \mid \Gamma_2$ and induction gives $e_1 \xrightarrow{\lambda} e_1'$ (because e_1 is not a value). Hence e reduces by rule Act-LetE*.

Rule E-Rec. Expression $\texttt{rec } x : T.v$ is a value

Rule E-App. Since $e = e_1\, e_2$ is not a value it must reduce. Distinguish two cases. When e_1 is not a value, by induction e_1 reduces and so does e (by rule ActAppL). When e_1 is a value, canonical forms (Theorem D.8) give a series of cases that we analyse in turn. When e_1 is an abstraction we further distinguish two cases. If e_2 is a value, then e reduces by rule Act-App. Otherwise monotonicity (Theorem D.5) gives $\Gamma_2 \subseteq \Gamma_1^{\mathbf{s}}$ hence $\Gamma_2^{\mathbf{s}}$. The induction hypothesis allow concluding that e reduces by rule Act-AppR. The case for $\texttt{rec}$ is similar to that of λ. When $e_1 = \texttt{fork}$, we further distinguish two cases. When e_2 is a value, e reduces by rule Act-Fork. Otherwise, using induction expression e reduces by rule Act-AppR. When $e_1 = \texttt{wait}$, we further distinguish two cases. When e_2 is a value, canonical forms give $e_2 = x$, hence e reduces by rule Act-Wait. Otherwise, monotonicity (Theorem D.5) places us in the induction hypothesis, hence e_2 reduces (because it is not a value), and e reduces by rule Act-AppR. The subcase for $\texttt{terminate}$ is similar. When $e_1 = \texttt{receive}[W][X]$, we further distinguish two cases. When e_2 is a value, canonical forms give $e_2 = x$, hence e reduces by rule Act-Rcv. Otherwise, monotonicity (Theorem D.5) places us in the induction hypothesis, hence e_2 reduces (because it is not a value), and e reduces by rule Act-AppR. The subcases for $\texttt{send}[W][X]$ and $\texttt{send}[W][X]\, u$ and $\texttt{select}\, C[\overline{W}]$ are similar.

Rule E-TApp. These cases are analogous to those of rule E-App.

Rule E-Pair. Distinguish three cases for $e = \langle e_1, e_2 \rangle$. When both e_1 and e_2 are values, e is a value. When e_1 is not a value, induction ensures that e_1 reduces and thus e has a transition by rule Act-PairL. Finally, when e_1 alone is a value, the premises to rule E-Pair include $\Delta \mid \Gamma_1^{\mathbf{s}} \vdash e_1 \Rightarrow T \mid \Gamma_2$. Monotonicity (Theorem D.5) gives $\Gamma_2 \subseteq \Gamma_1^{\mathbf{s}}$, hence $\Gamma_2^{\mathbf{s}}$ and we are in the induction hypothesis. Conclude with rule Act-PairR.

Rule E-Let. Similar to E-Let*.

Rule E-Match. In this case $e = \texttt{match } e' \texttt{ with } \{C_i\, x_i \to e_i\}_{i \in I}$. The premises to the rule include $\Delta \mid \Gamma_1^{\mathbf{s}} \vdash e' \Rightarrow S \mid \Gamma_2$ and $\mathrm{nrm}^+(S) = ?(P\, \overline{U}).S'$. Agreement (Theorem D.4) gives $\Delta \vdash S \Leftarrow \kappa$. Distinguish two cases. When e' is value, normalization of session types (Theorem D.2) and canonical forms (Theorem D.8) give $e' = x$ and e reduces by rule Act-Match. Otherwise, the result follows by induction followed by rule Act-MatchE.

Rule P-Exp. In this case $p = \langle e \rangle$. Distinguish two cases. When e is a value, canonical forms give $e = *$ and p is completed. Otherwise, by induction $e \xrightarrow{\lambda} e'$ and we further distinguish three cases according to label λ:

λ	LTS rule
$\texttt{fork}\, v$	Act-Fork
$(\nu xy)\, T$	Act-New
σ	Act-Session
β	Act-Beta

Rule P-Par. In this case $p = p_1 \mid p_2$. By induction each p_i is either completed, reduces or is a deadlock. We must analyse eight cases. When both p_i are completed, p is completed. When one of the p_i is a deadlock, then so is p. In case both p_i reduce, then p reduces by ACT-JoinR (or Act-JoinL). Otherwise, when p_1 reduces (and p_2 is completed), p reduces with rule Act-ParL. Finally, hen p_2 reduces (and p_1 is completed), p reduces with rule Act-ParR.

Rule P-New. In this case $p = (\nu xy)\, p_1$. The premise to the rule is $\Gamma, x : \mathrm{nrm}^+(T), y : \mathrm{nrm}^-(T) \vdash p_1$. Agreement (Theorem D.4) and the fact that duality is defined on session types alone establish that $(\Gamma, x : \mathrm{nrm}^+(T), y : \mathrm{nrm}^-(T))^{\mathsf{s}}$. By induction p_1 is completed, reduces or is a deadlock. Because completed processes are closed (Theorem D.9), p_1 cannot be completed. If p_1 is deadlocked, then so is p, by definition. It remains to check the case when p is not a deadlock and reduces. Suppose that $p_1 \xrightarrow{\pi} p_2$, with π in the progress set for x, y. Then p reduces according to the table below.

π	LTS rule
σ	Act-Session
$\texttt{fork}\, v$	Act-Fork
$(\nu ab)\, T$	Act-New
$x?v \mid y!v$	Act-Msg
$x?C \mid y!C$	Act-Bra
$x? \mid y!$	Act-Wait
$x?a \mid (\nu ab) y!a$	Act-CloseL
$x?b \mid (\nu ab) y!b$	Act-CloseR

□

E Embedding context-free session types

This section presents an informal investigation of two questions that relate AlgST and context-free session types (CFST [4]): can we map any AlgST program into an equivalent CFST program? And: can we map any CFST program to an equivalent AlgST program?

This investigation is necessarily informal because the features of AlgST and CFST do not line up. AlgST is a minimalist mostly linear version of System F with products, sessions, and algebraic protocols. CFST is also based on System F, but has a richer kind structure that enables unrestricted as well as linear features, variant and record types, and the full slate of context-free session primitives.

To enable a meaningful comparison, we focus on the linear fragment of CFST [4] using the notation of Costa et al. [16], restricted to pairs and ignoring De Bruijn indices. In AlgST, we only consider types that are not polymorphic in protocols because CFST has no equivalent. We add linear algebraic datatypes to cope with variant types in CFST.

Non-parameterized algebraic sessions into (linear) context-free sessions. The translation from algebraic session types that do not parameterize over protocol types (AlgST_0) to context-free

CFST to AlgST: Translation of the functional fragment $\boxed{[\![T : \mathsf{T}^{\mathsf{lin}}]\!] = T : \mathbf{T}}$

$$[\![\mathtt{unit}]\!] = \mathtt{Unit} \qquad [\![T \to U]\!] = [\![T]\!] \to [\![U]\!] \qquad [\![T \times U]\!] = [\![T]\!] \otimes [\![U]\!]$$

$$[\![\langle \ell : T_\ell \rangle_{\ell \in L}]\!] = X_L \text{ where } \mathtt{data}\ X_L = \{\ell\ [\![T_\ell]\!]\}_{\ell \in L}$$

$$[\![X : \kappa]\!] = X \text{ for } X \doteq T \text{ and } \kappa \neq \mathsf{S}^{\mathsf{lin}} \text{ where } \mathtt{data}\ X = \mathsf{Unfold}_X\ [\![T]\!]$$

$$[\![T : \mathsf{S}^{\mathsf{lin}}]\!] = !X_T.\mathsf{End}_! \text{ where } \mathtt{protocol}\ X_T = X_T\ (\!|T|\!) \text{ and } X_T \text{ is a fresh name.}$$

CFST to AlgST:Translation of the session typed fragment $\boxed{(\!|T : \mathsf{S}^{\mathsf{lin}}|\!) = \overline{T : \mathbf{P}}}$

$$(\!|\mathtt{skip}|\!) = \varepsilon \qquad (\!|!T|\!) = [\![T]\!] \qquad (\!|?T|\!) = -[\![T]\!] \qquad (\!|T;U|\!) = (\!|T|\!)(\!|U|\!)$$

$$(\!|\oplus\{\ell : T_\ell\}_{\ell \in L}|\!) = X_L \text{ where } \mathtt{protocol}\ X_L = \{\ell\ (\!|T_\ell|\!)\}_{\ell \in L}$$

$$(\!|\&\{\ell : T_\ell\}_{\ell \in L}|\!) = -X_L \text{ where } \mathtt{protocol}\ X_L = \{\ell\ (\!|\mathsf{dual}\ T_\ell|\!)\}_{\ell \in L}$$

$$(\!|X : \mathsf{S}^{\mathsf{lin}}|\!) = X \text{ for } X \doteq T \text{ where } \mathtt{protocol}\ X = \mathsf{Unfold}_X\ (\!|T|\!)$$

Fig. 14. Translation of linear context-free session types to algebraic session types

session types (CFST) is given by $\wr T \wr$, which is defined on types in normal form, only:

$$\wr \mathtt{Unit} \wr = \mathtt{unit} \quad \wr T \to U \wr = \wr T \wr \to \wr U \wr \quad \wr T \otimes U \wr = \wr T \wr \times \wr U \wr \quad \wr \forall \alpha : \mathbf{T}.T \wr = \forall \alpha : \mathbf{T}^{\mathbf{LIN}}.\wr T \wr$$

$$\wr \alpha \wr = \alpha \qquad \wr ?T.U \wr = ?\wr T \wr;\wr U \wr \qquad \wr !T.U \wr = !\wr T \wr;\wr U \wr \qquad \wr \mathsf{End}_! \wr = \mathtt{skip} \qquad \wr \mathsf{End}_? \wr = \mathtt{skip}$$

$$\wr ?P\ \overline{U}.S \wr = P_U;\wr S \wr \text{ where } P_U \doteq \&\{C_i : \wr \S(-(\overline{T_i[\overline{U}/\overline{\alpha}]})).\mathsf{End}_! \wr\}_{i \in I} \text{ for } \mathtt{protocol}\ \rho\ \overline{\alpha} = \{C_i\ \overline{T_i}\}_{i \in I}$$

$$\wr !P\ \overline{U}.S \wr = P_U;\wr S \wr \text{ where } P_U \doteq \oplus\{C_i : \wr \S(+(\overline{T_i[\overline{U}/\overline{\alpha}]})).\mathsf{End}_! \wr\}_{i \in I} \text{ for } \mathtt{protocol}\ \rho\ \overline{\alpha} = \{C_i\ \overline{T_i}\}_{i \in I}$$

Functional types and message exchanges are naturally mapped onto their counterparts. The end of a session is mapped to `skip`, whereas algebraic protocols are translated to type names defined as internal choices or branches, depending on whether they appear in sending or receiving positions. The choice labels coincide with the protocol constructors.

We claim that if $T \equiv_A U$, then $\wr T \wr \simeq \wr U \wr$.

Linear context-free sessions embedded into algebraic sessions. In this direction, we omit polymorphism to obtain a lighter, more perspicuous notation. Our comparative approach scales to the polymorphic setting, but at the price of high notational overhead.

The translation of the *linear* and *monomorphic* fragment of CFST to AlgST is split between the functional and the session typed fragments, which are mutually defined and presented in Figure 14. The translation of a context-free session type T to the algebraic setting is given by $[\![T]\!]$. The functional operators have natural counterparts in the algebraic side. When translated to the isorecursive setting, functional recursive types are represented by a datatype enclosed with an explicit unfold. Variant types are mapped to datatypes whose constructors are the respective labels. Session types are materialized by protocol definitions: each session type originates a protocol emerging from the conversion of a sequential composition of types into a sequence of (algebraic) types with kind

P. This conversion is governed by function $(\!|\cdot|\!)$ that maps `skip` to the empty sequence and uses polarities to enforce the direction of communication. In the session typed fragment, recursive types and choices are represented by protocol definitions. Type names are considered generative and we relax AlgST to admit overloaded constructor names that can be reused across protocol definitions.

It remains to define suitable adapters in the places where CFST relies on type equivalence of equirecursive types. To this end we generate isomorphisms as witnesses of equivalence proofs using the rule system proposed by Costa et al. [16]. The tricky part of constructing such an isomorphism is its extension into session types. For simplicity, we consider just one side of the isomorphism $\mathsf{iso} : [\![T]\!] \to [\![U]\!]$ where $T \simeq U$ are session types:

```
iso t = let (u, u') = new [[U]] in fork $ δ t u' ▷ u
```

This definition relies on a process $\delta : [\![T]\!] \to \mathtt{Dual}\ [\![U]\!] \to \mathtt{Unit}$ that consumes two (dual) channel ends and implements a forwarder that compensates for the differences between T and U. The witness processes δ are defined for each pair of equivalent session types, according to the rules. We sketch some witness processes in Table 1 and provide a more detailed analysis of the isomorphism.

For the sake of simplicity, in all cases in Table 1, we elide an initial administrative `match` and `select` step resulting from the definition of $[\![T : \mathsf{S}^{\mathsf{lin}}]\!]$. The case for $T \simeq U$ where $T = \mathtt{Skip}$ and $U = \mathtt{Skip}$ would be formally defined by:

```
δ t u = match u with {
  XU u → let t = select XT t in
         wait u
         ▷ terminate t }
```

Process δ `match`es with X_U on channel u and `select`s X_T on channel t, as prescribed by the protocol definitions resulting from $[\![\mathtt{Skip}]\!]$ (see Figure 14). According to the definition of $[\![\mathtt{Skip}]\!]$ and E-Match, we are left with type $\mathsf{End}_?$ on u and $\mathsf{End}_!$ on t, so we `wait` on u and `terminate` on t.

In general, δ inserts the explicit unfolds imposed by the translation to the isorecursive setting. We showcase our approach for the equivalence $Y \simeq U$ where type Y is defined by T, $Y \doteq T$. The definition of δ follows the equivalence rule of Costa et al. [16] as shown below. A witness process δ for the embedding $\mathsf{iso} : [\![Y]\!] \to [\![U]\!]$ builds on a process δ_1 witnessing $T \simeq U$ and is defined by:

```
δ y u = match u with {
   XU u → let y = select XY y in
          let t = select UnfoldY y in
          δ1 t u }
```

$$\frac{Y \doteq T \qquad T\ \mathsf{contr} \qquad \delta_1 : T \simeq U}{\delta : Y \simeq U}$$

As a result of the definition of $[\![Y : \mathsf{S}^{\mathsf{lin}}]\!]$, process δ `match`es with X_U on channel u and `select`s X_Y on channel y, as prescribed by the protocol definitions resulting from $[\![\mathtt{Y}]\!]$ (see Figure 14). Then, `select`s the explicit Unfold_Y introduced by $(\!|Y|\!)$ and uses δ_1 to consume the leftovers at t and u. In the case $X \simeq U$ where $X \doteq T$, we are provided with a witness δ_1 for $T \simeq U$. We `select` the explicit Unfold_X introduced by $(\!|X|\!)$ and then use δ_1 to consume the leftovers at t and u.

For the case of $!T;V \simeq\ !U;W$, we are provided not only with a witness δ_1 to consume the remainder of channels v and w, but also with a function $\theta : [\![T]\!] \to [\![U]\!]$ that witnesses the isomorphism resulting from $T \simeq U$. The functions witnessing the isomorphism between types in the functional fragment are presented in Table 2. Given δ_1 and θ, after the initial administrative `match` and `select`, process δ `receive`s u on channel w, transforms u with θ^{-1} and sends the result through channel v. Process δ_1 consumes the remainder of v and w.

To witness the isomorphism corresponding to $!T;V \simeq\ !U$ where V is a terminated session (see Costa et al. [16] for further details), we are given a process σ that consumes the explicit unfolds

resulting from the isorecursive interpretation of V. The *consumer* processes σ for terminated sessions are presented in Table 3.

The other cases in Tables 1 to 3 follow a similar reasoning. The omitted cases for monomorphic types also fit the same approach. For the polymorphic fragment, we would need to consider a *reader monad* where one could store a map from type variables to the processes that consume the corresponding channels or transform the corresponding values, depending on whether the type variables are of kind **s** or not. In the axiom for polymorphic (non-session typed) variables $n \simeq n$, for instance, we could think of θ as the identity function, but we wouldn't know which type to assign to the bound variable: $\theta = \lambda x : ?.x$. Whereas to consume two channels typed with a polymorphic session-typed variable, we would need to know how to consume both endpoints. Processes to consume or transform values typed with polymorphic variables would need to be provided to δ. For the translation of types in the polymorphic fragment, we would need to collect the free variables occurrig in the type and use a parametrized `data` or `protocol` definition:

$$[\![X : \kappa]\!] = X\ \overline{\alpha} \text{ for } X \doteq T \text{ and } \kappa \neq \mathsf{S}^{\mathsf{lin}} \text{ where } \texttt{data}\ X\ \overline{\alpha} = \mathsf{Unfold}_X\ [\![T]\!] \text{ and } \mathrm{fv}(T) = \overline{\alpha}$$

$$(\!|X : \mathsf{S}^{\mathsf{lin}}|\!) = X\ \overline{\alpha} \text{ for } X \doteq T \text{ where } \texttt{protocol}\ X\ \overline{\alpha} = \mathsf{Unfold}_X\ (\!|T|\!) \text{ and } \mathrm{fv}(T) = \overline{\alpha}$$

Although we believe that polymorphic types do not pose any problem to this analysis, we decided to keep the notation simple and focus on the monomorphic fragment.

In this analysis we only provided witnesses for the isomorphism in one direction. For the other direction of the isomorphism, we could follow a similar approach for defining $\mathrm{iso}^{-1} : [\![U]\!] \to [\![T]\!]$ and then prove that iso and iso^{-1} compose giving the identity.

Equivalence rule $T \simeq U : \mathsf{S}^{\mathsf{lin}}$	Isomorphism witness $[\![T]\!] \to \texttt{Dual}\ [\![U]\!] \to \texttt{Unit}$
$\delta : \mathsf{Skip} \simeq \mathsf{Skip}$	`δ t u = wait u` `  ▷ terminate t`
$\dfrac{X \doteq T \qquad T\ \mathsf{contr} \qquad \delta_1 : T \simeq U}{\delta : X \simeq U}$	`δ x u = let t = select UnfoldX x in` `  δ1 t u`
$\dfrac{X \doteq T \qquad T\ \mathsf{contr} \qquad \delta_1 : T;V \simeq U}{\delta : X;V \simeq U}$	`δ x u = let t = select UnfoldX x in` `  δ1 t u`

$\dfrac{\theta : T \simeq U \qquad \delta_1 : V \simeq W}{\delta : !T;V \simeq !U;W}$	`δ v w = let (u, w) = receive w in`
	`  let t = θ⁻¹ u`
	`  let v = send t v in`
	`  δ1 v w`

$\dfrac{\theta : T \simeq U \qquad \sigma : V\checkmark}{\delta : !T;V \simeq !U}$	`δ t u = let (u', u) = receive u in`
	`  let t' = θ⁻¹ u'`
	`  let v = send t' t in`
	`  let v = σ v in`
	`  wait u`
	`  ▷ terminate v`

$\dfrac{\theta : T \simeq U}{\delta : !T \simeq !U}$	`δ t u = let (u', u) = receive u in`
	`  let t' = θ⁻¹ u' in`
	`  let t = send t' t in`
	`  wait u`
	`  ▷ terminate t`

$\dfrac{\delta_\ell : T_\ell \simeq U_\ell \qquad (\forall \ell \in L)}{\delta : \oplus\{\ell : T_\ell\}_{\ell\in L} \simeq \oplus\{\ell : U_\ell\}_{\ell\in L}}$	`δ t u = match u with {`
	`  k u → let t = select k t in`
	`  δk t u }`

$\dfrac{\delta_1 : \oplus\{\ell : T_\ell;U\}_{\ell\in L} \simeq V}{\delta : \oplus\{\ell : T_\ell\}_{\ell\in L};U \simeq V}$	`δ t u = δ1 t u`
$\dfrac{\delta_1 : T \simeq U}{\delta : \mathsf{Skip};T \simeq U}$	`δ t u = δ1 t u`

Table 1. Processes that consume two dual session endpoints.

Equivalence rule $T \simeq U : \kappa$ with $\kappa \neq \mathsf{S}^{\mathsf{lin}}$	Isomorphism witness $\theta : [\![T]\!] \to [\![U]\!]$
$\theta : ()_{\mathsf{lin}} \simeq ()_{\mathsf{lin}}$	`θ = λ x:Unit.x`
$\dfrac{\theta_1 : T \simeq U \qquad \theta_2 : V \simeq W}{\theta : T \to V \simeq U \to W}$	`θ t = λ x:⟦U⟧.θ2 $ t $ θ⁻¹ x`
$\dfrac{\theta_1 : T \simeq U \qquad \theta_2 : V \simeq W}{\theta : T \otimes V \simeq U \otimes W}$	`θ t = <θ1 $ fst t,θ2 $ snd t>`

$\dfrac{X \doteq T \qquad T\ \mathsf{contr} \qquad \theta_1 : T \simeq U}{\theta : X \simeq U}$	`θ x = let t = select UnfoldX x in`
	`let u = θ1 t in`
	`u`

Table 2. Transformation of non-session typed expressions.

Is terminated predicate $T : \mathsf{S}^{\mathsf{lin}}$	Witness $\sigma : [\![T]\!] \to \mathsf{End}_!$
$\sigma : \mathsf{Skip}\checkmark$	`σ t = t`
$\dfrac{\sigma_1 : T\checkmark \qquad \sigma_2 : U\checkmark}{\sigma : T;U\checkmark}$	`σ t = let t' = σ1 t in` `σ2 t'`
$\dfrac{T \doteq U \qquad \sigma_1 : T\checkmark}{\sigma : X\checkmark}$	`σ x = let t = select UnfoldX x in` `σ1 t`

Table 3. Processes that consume the explicit unfolds created by the translation.